



UNIVERSIDAD DE EXTREMADURA

ESCUELA POLITÉCNICA

INGENIERÍA INFORMÁTICA EN INGENIERÍA DEL
SOFTWARE

TRABAJO FIN DE GRADO

**Optimización de las emisiones de CO₂ en
redes definidas por software mediante
algoritmos bioinspirados**

Santiago Rodríguez-Arias Martínez-Berná
Convocatoria, 2026



Escuela Politécnica

UNIVERSIDAD DE EXTREMADURA

ESCUELA POLITÉCNICA

INGENIERÍA INFORMÁTICA EN INGENIERÍA DEL
SOFTWARE

TRABAJO FIN DE GRADO

**Optimización de las emisiones de CO₂ en
redes definidas por software mediante
algoritmos bioinspirados**

Autor: Santiago Rodríguez-Arias Martínez-Berná

Tutor: Jaime Galán Jiménez

Co-Tutor: José Antonio Gómez de la Hiz

Agradecimientos

Creo que es injusto atribuirme todo el mérito de llegar hasta aquí. Hay quien podría decir que es gracias a mi esfuerzo y dedicación que he conseguido alcanzar la meta. Y tendría razón. Aunque solo en parte. Es gracias a otras personas que, a lo largo de mi vida y de la carrera, me han apoyado y empujado a continuar hacia adelante.

En primer lugar, quiero darle las gracias a mis padres, Mariano y Mónica, por su apoyo incondicional a lo largo de los años. Cuando llevaba ya casi 5 años en Ingeniería Electrónica y Automática Industrial, les dije que me quería cambiar de carrera, y después de poner un total de 0 pegas, me dijeron que adelante. Saber que tenía todo su apoyo en un cambio tan grande, me calmó por completo y pude empezar de cero con fuerzas. A raíz de sus años de experiencia, tanto en la universidad como en sus trabajos, me han brindado consejos muy valiosos.

Agradecer también a mis tutores, Jaime Galán Jiménez y José Antonio Gómez de la Hiz, por su dedicación y por permitirme realizar el TFG con ellos. Ambos me han enseñado lo interesante que pueden llegar a ser las redes, en especial en temas aparentemente aburridos como la reducción de la huella de carbono. Gracias por la paciencia, los consejos y el tiempo empleado para desarrollar este trabajo.

Y por último, aunque no menos importante, darle las gracias a mi amigo Rubén Serrano Izquierdo por las píldoras de información y los datos curiosos sobre redes y temas de *backend*. Fue quien abrió la puerta a mi interés por las redes y a plantearme un TFG relacionado con este tema.

Muchas gracias a todos.

Resumen

A día de hoy, las redes de comunicaciones son un componente esencial en la vida diaria de las personas y en el funcionamiento de las empresas. El aumento del tráfico de datos ha crecido de forma exponencial, llegando a multiplicarse por 80 desde 2007 hasta 2023 [1]. Aunque el crecimiento del consumo energético no es proporcional al aumento del tráfico de datos, pues apenas se ha duplicado, esto no significa que no haya un impacto ambiental significativo. Se estima que las redes de comunicaciones representan aproximadamente el 1% del consumo eléctrico mundial y alrededor del 1,4% de las emisiones globales de CO_2 . Y estos datos no van a hacer más que aumentar debido al auge de las Inteligencias Artificiales (IA) y los grandes centros de procesamiento de datos. Por tanto, es fundamental buscar soluciones que permitan optimizar el uso de las redes para reducir su impacto ambiental.

En este trabajo se propone una solución basada en redes definidas por software (SDN) y algoritmos bioinspirados para optimizar las emisiones de CO_2 asociadas al uso de las redes de comunicaciones. Más concretamente, se parte de los avances expuestos en [2], donde los autores proporcionan una fórmula que permite calcular las emisiones de CO_2 de las redes cableadas en función del tráfico de datos y el consumo energético de los dispositivos de red. Con esta fórmula en mente, se ha modelado una función de optimización que se usará en un algoritmo de optimización por enjambre de partículas (PSO) para encontrar la configuración de red que minimice las emisiones de CO_2 .

Específicamente, en este trabajo se presentan dos versiones del algoritmo, en una de ellas se deja de evaluar una solución si esta no cumple alguna de las restricciones, mientras que en la otra se penalizan las soluciones que no cumplen dichas restricciones. En cuanto a

las pruebas, se estudian cómo diferentes técnicas de enrutamiento, el aumento del tráfico y la escalabilidad de la topología de red afectan a las emisiones de CO_2 .

Palabras clave: SDN, enrutamiento consciente de carbono.

Abstract

Nowadays, communication networks are an essential component of people's daily lives and business operations. Data traffic has grown exponentially, increasing by a factor of 80 between 2007 and 2023 [1]. Although the growth in energy consumption has not been proportional to the increase in data traffic, having only doubled, this does not mean that there is no significant environmental impact. Communication networks are estimated to account for approximately 1% of global electricity consumption and around 1.4% of global CO_2 emissions. Moreover, these figures are expected to continue rising due to the growth of Artificial Intelligence (AI) and large-scale data processing centers. Therefore, it is essential to seek solutions that optimize the use of communication networks in order to reduce their environmental impact.

This work proposes a solution based on Software-Defined Networking (SDN) and bio-inspired algorithms to optimize the CO_2 emissions associated with the use of communication networks. More specifically, it builds upon the advances presented in [2], where the authors provide a formula to calculate the CO_2 emissions of wired networks as a function of data traffic and the energy consumption of network devices. Based on this formula, an optimization function has been modeled to be used within a Particle Swarm Optimization (PSO) algorithm in order to find the network configuration that minimizes CO_2 emissions.

Specifically, this work presents two versions of the algorithm. In one version, a solution stops being evaluated if it violates any of the constraints, whereas in the other, solutions that do not satisfy the constraints are penalized. Regarding the experiments, the study analyzes how different routing techniques, increased traffic, and the scalability of the

network topology affect CO_2 emissions.

Keywords: SDN, carbon-aware routing.

Índice general

1. Introducción	1
2. Objetivos y Contribuciones	5
2.1. Objetivos	5
2.2. Contribuciones	7
3. Estado del Arte	8
3.1. Redes Cableadas y Emisiones de Carbono	8
3.1.1. Consumo en redes	8
3.1.2. Modelo de emisiones	8
3.2. Estrategias de <i>green networking</i>	9
3.2.1. <i>Software Defined Networks</i>	9
3.2.2. Enrutamiento Consciente de Carbono	9
3.3. Técnicas de Optimización	9
3.3.1. Métodos clásicos	9
3.3.2. Metaheurísticas	9
3.3.3. Métodos basados en aprendizaje automático	10
3.4. <i>Particle Swarm Optimization</i>	10
3.5. No sé qué más	10
4. Metodología	11
4.1. Desarrollo en Espiral	11
4.2. Reuniones Bisemanales	13

4.3. Diagrama de Gantt	14
5. Descripción del Algoritmo Propuesto	16
5.1. Elección de la Técnica de Optimización	16
5.2. <i>Particle Swarm Optimization</i>	18
5.3. Parámetros de PSO	19
5.3.1. Partícula	19
5.3.2. Función objetivo	20
5.3.3. Velocidad	20
5.3.4. Posición	20
5.3.5. Coeficiente de inercia	21
5.3.6. Coeficiente cognitivo	21
5.3.7. Coeficiente social	22
5.3.8. Número de vecinos	22
5.3.9. Dimensiones del espacio de búsqueda	22
5.4. Aproximación al Problema	23
5.4.1. Estrategias de manejo de restricciones	24
5.5. Implementación y Desarrollo del Algoritmo	25
5.5.1. Configuración de las partículas	25
5.5.2. Iteraciones	28
5.5.3. Coeficientes cognitivo y social	29
5.5.4. Inercia, vecinos y dimensiones del espacio de búsqueda	30
5.5.5. Función objetivo y pseudocódigo	31
6. Pruebas y Resultados	35
6.1. Detalles de la Implementación	35
6.2. Topologías Utilizadas	37
6.3. Definición de las Pruebas	38
6.4. Resultados para Abilene	38
6.4.1. Emisiones de referencia	39

6.4.2.	Emisiones después del apagado de enlaces	40
6.4.3.	Tiempos de ejecución	44
6.4.4.	Porcentaje de enlaces apagados	45
6.4.5.	Ocupación de los enlaces tras el apagado	46
6.5.	Resultados para Nobel y Geant	47
6.5.1.	Emisiones de cada matriz de tráfico	47
6.5.2.	Emisiones después del apagado de enlaces	49
6.5.3.	Tiempos de ejecución	52
6.5.4.	Porcentaje de enlaces apagados	53
6.5.5.	Ocupación de los enlaces tras el apagado	55
6.6.	Análisis de Germany y Limitaciones Encontradas	56
7.	Conclusiones y Trabajos Futuros	59
7.1.	Conclusiones	59
7.2.	Trabajos Futuros	60
	Anexos	62
	A. Acrónimos	63
	Bibliografía	64

Índice de tablas

6.1. Detalles topologías	37
------------------------------------	----

Índice de figuras

1.1. Esquema de redes SDN	3
4.1. Desarrollo en espiral	12
4.2. Diagrama de Gantt	14
5.1. Diagrama de flujo de PSO	18
5.2. Función sigmoide	23
5.3. Ejemplo sencillo PSO	26
5.4. Barrido partículas Abilene	27
5.5. Barrido partículas Abilene con inicialización controlada	28
5.6. Barridos iteraciones Abilene	29
5.7. Esquema consumo router	32
6.1. Topología Abilene	39
6.2. Emisiones máximas Abilene	40
6.3. Resultados Abilene PSO 200p 1200i 20runs	40
6.4. Resultados Abilene PSO 200p 1200i 100runs	41
6.5. Resultados Abilene PSO VCH 200p 1200i 20runs	42
6.6. Resultados Abilene PSO VCH 100p 600-1200i 20runs	43
6.7. Comparacion de emisiones de Abilene	44
6.8. Tiempos ejecución Abilene	45
6.9. Porcentaje enlaces apagados Abilene	46
6.10. Uso de enlaces Abilene	47
6.11. Emisiones máximas Nobel	48

6.12. Emisiones máximas Geant	49
6.13. Resultados Geant y Nobel PSO normal 200p 1200i 20runs	50
6.14. Resultados Geant y Nobel PSO VCH 200p 1200i 20runs	51
6.15. Resultados Geant y Nobel PSO VCH 100p 600i 20runs	51
6.16. Comparacion de emisiones de Geant y Nobel	52
6.17. Tiempos ejecución Geant	53
6.18. Tiempos ejecución Nobel	53
6.19. Porcentaje enlaces apagados Geant	54
6.20. Porcentaje enlaces apagados Nobel	55
6.21. Uso de enlaces Geant	55
6.22. Uso de enlaces Nobel	56
6.23. Barrido iteraciones Germany	57

Capítulo 1

Introducción

Cuando en 1969, J. C. R. Licklider, un científico de la computación estadounidense, consiguió conectar el Instituto de Investigación de Stanford con la Universidad de California en Los Ángeles [3], no podía imaginar en lo que llegaría a convertirse esa conexión. En ese momento, la idea de compartir recursos informáticos entre dos instituciones parecía revolucionaria, pero hoy en día, esa conexión es la base de lo que se conoce como Internet.

Desde entonces, Internet no ha hecho más que crecer y evolucionar, especialmente en los últimos años. Esto ha generado un interés creciente en mejorar la calidad del servicio (*QoS*) optimizando aspectos como la latencia, el ancho de banda y la fiabilidad, pues Internet se ha convertido en una parte fundamental de las vidas y del trabajo de las personas. Sin embargo, cuanto más se usa Internet, más recursos se necesitan para mantenerlo funcionando, lo que ha llevado a un aumento en el consumo de energía y, por ende, a un impacto ambiental significativo.

Aunque parezca que la preocupación por la sostenibilidad es algo reciente, la verdad es que lleva existiendo desde hace décadas. Por ejemplo, el 14 de marzo de 2001, el Parlamento Europeo adoptó una resolución sobre el impacto del consumo de energía y la necesidad de reducir este consumo en un 2.5 % anual, entre otras propuestas [4]. Otros estudios, como el de Paul J.M. Havinga y Gerard J.M. Smit [5], han analizado el consumo de energía en redes inalámbricas y dispositivos móviles, destacando la importancia de la

eficiencia en estos escenarios.

Es lógico mencionar el impacto ambiental cuando se habla de consumo energético, pues el término *intensidad de carbono* define la relación existente entre las emisiones de carbono producidas al generar un kWh de energía. Estas emisiones dependen en gran medida de las fuentes de energía utilizadas: como comenta José Antonio Gómez de la Hiz en su estudio [6], producir energía con carbón o gas natural emiten 950 y 450 gCO_2/kWh , respectivamente, mientras que usando fuentes renovables como el aire, las emisiones se reducen hasta los 15 gCO_2/kWh . Por tanto, si se consigue enrutar el tráfico por nodos cuyas fuentes de energía son "verdes" y reducir así la carga de aquellos enlaces pertenecientes a dispositivos ubicados en zonas de altas emisiones, se podría alcanzar una reducción de las emisiones totales de carbono de la red. Si, además, este enrutado se combina con un apagado eficiente de enlaces, se podrían reducir más las emisiones.

Uno de los grandes avances en redes que ha permitido mejorar la eficiencia energética es el desarrollo de las redes definidas por software (SDN) [7]. Las redes SDN y las redes tradicionales funcionan, esencialmente, de la misma manera: los dispositivos de red, como routers y switches, se encargan de dirigir el tráfico de datos hacia su destino. La diferencia fundamental entre ambos tipos de redes radica en cómo se gestionan y controlan estos dispositivos. En las redes tradicionales, cada dispositivo de red tiene integrado tanto el plano de control como el plano de datos, lo que significa que cada dispositivo toma sus propias decisiones sobre cómo manejar el tráfico. Por otro lado, las redes SDN separan el plano de control de los dispositivos y lo centralizan en lo que se denomina un controlador SDN. Este es el responsable de tomar decisiones sobre cómo debe dirigirse el tráfico y enviar estas instrucciones a los dispositivos de red, que se limitan a ejecutar las órdenes recibidas.

En la Figura 1.1 se muestra un sencillo esquema de cómo funcionan las redes SDN. En este esquema, se puede observar cómo el controlador SDN se comunica con los dispositivos de red para indicarles cómo deben manejar el tráfico de datos. Si bien esta separación entre el plano de control y el plano de datos puede parecer un cambio menor, en realidad tiene un impacto significativo en la eficiencia energética de las redes. Al centralizar el control,

las redes SDN pueden optimizar el uso de los recursos de red en función del algoritmo que esté ejecutándose en el controlador. Por ejemplo, si se quisiera reducir todo lo posible la carga de los enlaces, basta con definir e implementar un algoritmo que se centre en optimizar la redirección del tráfico para repartir equitativamente entre todos los enlaces todos los flujos de datos.

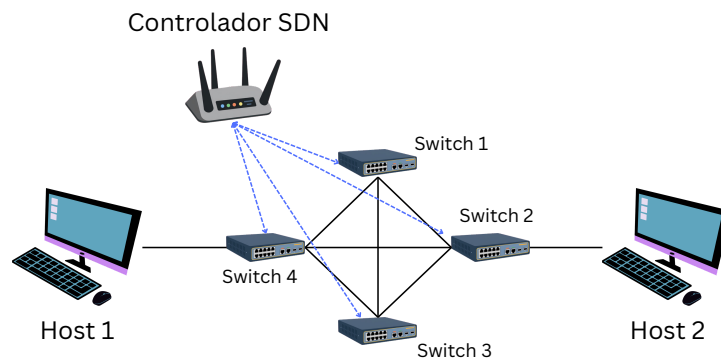


Figura 1.1: Representación gráfica de una red SDN.

Fuente: Elaboración propia.

El objetivo de este trabajo es reducir las emisiones de carbono de las redes cableadas SDN, y para ello, se implementará un framework que permita al controlador SDN realizar un enrutamiento consciente del carbono a la vez que realiza un apagado eficiente de enlaces, como se ha comentado con anterioridad. Específicamente, si los enlaces que se apagan son aquellos que se encuentran en zonas de altas emisiones de carbono, se conseguiría reducir las emisiones totales de la red. Pero no basta solamente con apagar dichos enlaces, hay que tener en cuenta dos factores muy importantes: la conectividad de la red y la carga de los enlaces. De nada sirve reducir las emisiones si a cambio se aumenta considerablemente la latencia al sobrecargar los enlaces o se deja desconectado un nodo por apagar los enlaces que lo conectan a la red. Es aquí donde entra en juego el concepto de enrutamiento consciente de carbono. Este término se asemeja a la idea detrás del algoritmo que publicó Edsger W. Dijkstra en 1956, aunque en lugar de buscar

el camino más corto, la idea es enrutar el tráfico a través del camino más "verde".

Al estar hablando de un problema de optimización, existen muchas técnicas que podrían ser aplicadas para resolverlo. Como se busca una solución que ofrezca resultados lo suficientemente buenos en un tiempo razonable, las opciones a considerar se limitan bastante. En este caso, se ha optado por usar un algoritmo metaheurístico de optimización por enjambre de partículas (PSO), pues es un algoritmo que se ha demostrado eficaz en problemas de optimización similares y que, además, es relativamente sencillo de implementar.

En definitiva, los capítulos siguientes detallan el diseño, implementación y evaluación de dicho algoritmo, desde la elección y adaptación de PSO hasta el análisis de los resultados obtenidos en diferentes topologías de red, con el fin de demostrar que la combinación del enrutamiento consciente de carbono y el apagado eficiente de enlaces permite reducir las emisiones de una red cableada SDN.

El resto de la memoria se estructura de la siguiente forma: en el Capítulo 2 se habla sobre los objetivos generales y específicos, así como de las contribuciones realizadas. A lo largo del Capítulo 3 se explican diferentes conceptos y técnicas relacionados con el problema que se está tratando de resolver, mientras que en el Capítulo 4 se detalla la metodología seguida para la realización de este trabajo. El algoritmo que finalmente se ha desarrollado se detalla en el Capítulo 5, mientras que en el Capítulo 6 se presentan los resultados obtenidos de las pruebas realizadas. Finalmente, en el Capítulo 7, se presentan las conclusiones que se han sacado en claro de los resultados obtenidos, así como las líneas de trabajo futuras que se podrían seguir a partir de este trabajo.

Capítulo 2

Objetivos y Contribuciones

En este capítulo se detalla el objetivo principal planteado para el desarrollo de este trabajo, así como los objetivos específicos en los que se desglosa para poder alcanzar dicho propósito. Además, se presentan las contribuciones más relevantes que se han aportado tras la realización de este Trabajo de Fin de Grado (TFG).

2.1. Objetivos

El principal objetivo de este trabajo es proponer una solución que permita reducir las emisiones de CO_2 en redes cableadas provenientes del consumo energético de los dispositivos de red. La solución se basa en el diseño, implementación y prueba de un framework que realiza conjuntamente el enrutamiento consciente de carbono y el apagado eficiente de enlaces, para así modificar la configuración de la red y que el consumo energético de sus dispositivos se centre en zonas de bajas emisiones y, por tanto, se reduzca la huella de carbono asociada a la infraestructura. Para ello, se han definido los siguientes objetivos específicos:

1. Análisis de datos y modelado de la red.
 - Identificar y procesar las fuentes de datos necesarias para modelar las redes cableadas y sus emisiones de CO_2 .



- Representar la topología de red mediante estructuras de datos adecuadas, como matrices o listas, con las que trabajar a partir de archivos *csv* y *JSON*.

2. Estudio del apagado de enlaces.

- Investigar el funcionamiento de la técnica de apagado de enlaces para la reducción del consumo energético en redes SDN.
- Estudiar criterios y restricciones a tener en cuenta para el apagado de enlaces sin comprometer la conectividad o el rendimiento de la red.
- Analizar el impacto del apagado de enlaces sobre el enrutamiento del tráfico de datos y las emisiones de CO_2 .

3. Estudio de soluciones de enrutado conscientes de carbono.

- Estudiar diversas técnicas de encaminamiento del tráfico de datos.
- Analizar diferentes propuestas de enrutamiento conscientes de carbono.
- Estudiar cómo incorporar la información de las emisiones en el proceso de enrutamiento.

4. Desarrollo del algoritmo de optimización.

- Diseñar e implementar un algoritmo de PSO que se adapte a las características del problema planteado.
- Integrar técnicas de modularización y reutilización de código para facilitar el desarrollo y la escalabilidad del algoritmo de cara a futuras mejoras.

5. Validación y análisis de resultados.

- Hacer uso de topologías reales para evaluar el rendimiento del algoritmo desarrollado.
- Analizar los resultados obtenidos para determinar la efectividad del algoritmo en la reducción de emisiones de carbono.



2.2. Contribuciones

Tras la realización de este trabajo, se han aportado las contribuciones que a continuación se detallan:

- Propuesta de un algoritmo de encaminamiento verde. Se ha diseñado e implementado un algoritmo de PSO adaptado a espacios de búsqueda discretos, integrando tanto el enrutamiento consciente de carbono como el apagado eficiente de enlaces.
- Uso de topologías de red reales. La validación del algoritmo se ha llevado a cabo usando las siguientes topologías: Abilene, Geant, Nobel-Germany y Germany50. Esto permite que los experimentos se ajusten más a la realidad y poder extrapolar los resultados obtenidos a escenarios de red reales.
- Pruebas y análisis. Se ha realizado una evaluación de los resultados obtenidos, demostrando que la combinación del enrutamiento consciente de carbono con el apagado eficiente de enlaces, reduce de forma significativa la huella de carbono de la infraestructura de red.

Capítulo 3

Estado del Arte

Una de las tareas más importantes a la hora de desarrollar un proyecto es realizar una investigación exhaustiva y así determinar el marco teórico bajo el que se va a desarrollar el proyecto. En este capítulo se presenta un estudio en profundidad de los puntos más importantes relacionados con el proyecto, como son los algoritmos de optimización, las técnicas de enrutamiento o las métricas de evaluación.

3.1. Redes Cableadas y Emisiones de Carbono

La idea de esta sección es hablar sobre las principales fuentes de consumo de las redes e intensidad de carbono.

3.1.1. Consumo en redes

Redes y fuentes de consumo de las redes.

3.1.2. Modelo de emisiones

Modelo de energía, intensidad de carbono.



3.2. Estrategias de *green networking*

Aquí se puede hablar sobre la diferencia entre redes tradicionales y redes SDN (aunque ya se ha hablado de ello en la introducción). También se puede hablar sobre diferentes técnicas de enrutado centradas en el carbono (o adaptadas al carbono) o sobre el enfoque de redes conscientes de carbono.

3.2.1. *Software Defined Networks*

Explicar las redes SDN y su papel en todo esto.

3.2.2. Enrutamiento Consciente de Carbono

- Dijkstra usando las emisiones como peso de los enlaces
- KSP usando las emisiones como peso de los enlaces

3.3. Técnicas de Optimización

Creo que aquí es buena idea hablar sobre las diferentes técnicas de optimización y sus ventajas y desventajas. Pero no sé si estaría pisando el contenido de la sección 5.1

3.3.1. Métodos clásicos

- ILP - MILP
- Heurísticas deterministas (voraces)

3.3.2. Metaheurísticas

- Algoritmos genéticos
- PSO
- Ant Colony o Simulated Annealing



3.3.3. Métodos basados en aprendizaje automático

- Deep Learning (importante dejar claro que DL no optimiza, sino que aprende y toma decisiones)

3.4. *Particle Swarm Optimization*

No sé si sería repetitivo tener esta sección aquí, ya se habla de PSO en la sección 5.2 y 5.3. Puede que sea mejor que la sección sea "PSO aplicado a redes SDN" o "PSO aplicado a la reducción de emisiones".

3.5. No sé qué más

Pues eso, noto que faltaría una sección para cerrar el capítulo, pero no estoy seguro sobre qué. A lo mejor es buena idea que este apartado sea como el de Amanda, que habla sobre las limitaciones existentes como la variabilidad en la generación de energía o la falta de transparencia (no se saben modelos exactos de routers para poder realizar simulaciones más exactas).

Capítulo 4

Metodología

Para el desarrollo de este trabajo, se ha seguido una metodología basada en el desarrollo en espiral [8], la cual se ha complementado con una estrategia de reuniones bisemanales para evaluar el desarrollo del proyecto y resolver las dudas que a lo largo de este han surgido.

4.1. Desarrollo en Espiral

La metodología del desarrollo en espiral fue propuesta por Barry Boehm en 1986 y se centra en la idea de avanzar en ciclos o iteraciones, donde en cada vuelta a la espiral se aumenta el nivel de desarrollo y la complejidad. A diferencia de un modelo en cascada donde todo se hace una sola vez en orden, en el desarrollo en espiral se repiten las mismas fases varias veces, refinando y ampliando el producto en cada ciclo. Normalmente, cada ciclo pasa por cuatro fases:

- Planificación: se definen los objetivos del ciclo.
- Análisis de riesgos: se identifican los problemas potenciales y se toman decisiones para minimizarlos.
- Ingeniería: se diseña, implementa y prueba el incremento correspondiente.
- Evaluación: el cliente o tutor revisa el resultado

La gran ventaja que tiene este modelo frente a otros es el énfasis en los riesgos: antes de invertir tiempo en desarrollar algo, se analiza si tiene sentido hacerlo y cómo evitar que falle. Es especialmente útil en el caso de un TFG, pues al principio no se sabe muy bien qué hacer ni cómo. El resultado de cada ciclo suele ser un prototipo o versión funcional cada vez más completa hasta llegar al producto final en el último ciclo.

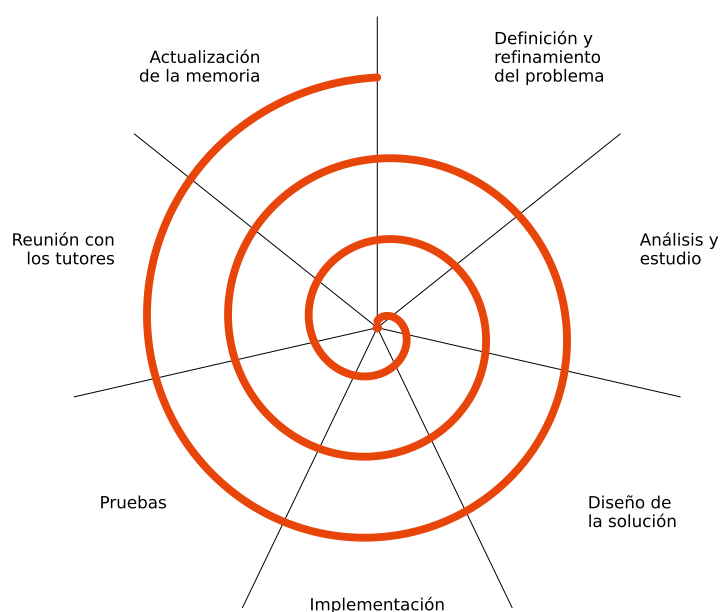


Figura 4.1: Estructura del desarrollo en espiral.
Fuente: Elaboración propia.

En la Figura 4.1 se muestra el diagrama del modelo en espiral seguido en este trabajo, donde cada vuelta representa un ciclo completo de desarrollo. Las fases por las que pasa cada ciclo se explican con más detalle a continuación:

- Definición y refinamiento del problema. En el primer ciclo se define el problema inicial. En ciclos posteriores, esta fase consiste en refinar dicho problema a partir de los resultados de la iteración anterior.
- Análisis y estudio. A partir del refinamiento en la fase anterior, se analizan los nuevos requisitos y se estudia cómo abordarlos.



- **Diseño de la solución.** En esta fase se procede a diseñar cómo ajustar la solución al trabajo en base a la información recopilada en la fase de análisis y estudio.
- **Implementación.** Una vez determinado cómo ajustar la solución al trabajo, se realiza la implementación en código.
- **Pruebas.** Para poder evaluar la solución implementada, en esta fase se realizan diferentes pruebas que ofrezcan resultados para poder analizar.
- **Reunión con los tutores.** Se realiza una reunión con los tutores para compartir el progreso, pruebas y resultados, de esta manera se obtienen nuevos puntos de vista, se resuelven dudas, se ponen sobre la mesa nuevas vías de trabajo y se establece la dirección de la siguiente iteración.
- **Actualización de la memoria.** A lo largo del ciclo, y antes de pasar a la siguiente iteración, se documentan los avances para, más adelante, redactar la versión final de la memoria.

4.2. Reuniones Bisemanales

Para mantener un ritmo constante en el desarrollo del trabajo y asegurar un seguimiento eficiente del proyecto, se decidió establecer reuniones con los tutores cada dos semanas. Esto ha permitido revisar tanto el estado como los avances del trabajo, además de resolver dudas y refinar el alcance. De la misma manera, las reuniones han servido para identificar problemas, explorar posibles soluciones y acordar la dirección de los siguientes pasos antes de comenzar cada nueva iteración.

Esta estrategia de reuniones bisemanales ha sido muy valiosa para mantener el enfoque del trabajo, evitando desviaciones y corrigiendo el desarrollo cuando ha sido necesario. Además, la amplia experiencia de los tutores ha enriquecido el trabajo con puntos de vista que no se hubiesen tenido en cuenta en un desarrollo más independiente.



4.3. Diagrama de Gantt

El diagrama de Gantt es una herramienta gráfica con el objetivo de mostrar el tiempo dedicado a las diferentes tareas de un proyecto a lo largo de un tiempo total determinado. Es este el motivo por el cual se ha optado por elegir esta herramienta para representar el tiempo dedicado a cada parte del proyecto.

Dado que durante el primer cuatrimestre del curso 2025-2026 se tenía previsto cursar seis asignaturas, el tiempo disponible para dedicar al TFG sería limitado. Por este motivo, se habló con el tutor, Jaime Galán, sobre la posibilidad de adelantar el inicio del trabajo a enero-febrero de 2025, una vez finalizados los exámenes, permitiendo así avanzar en las fases iniciales antes de que comenzara el período de mayor carga académica.

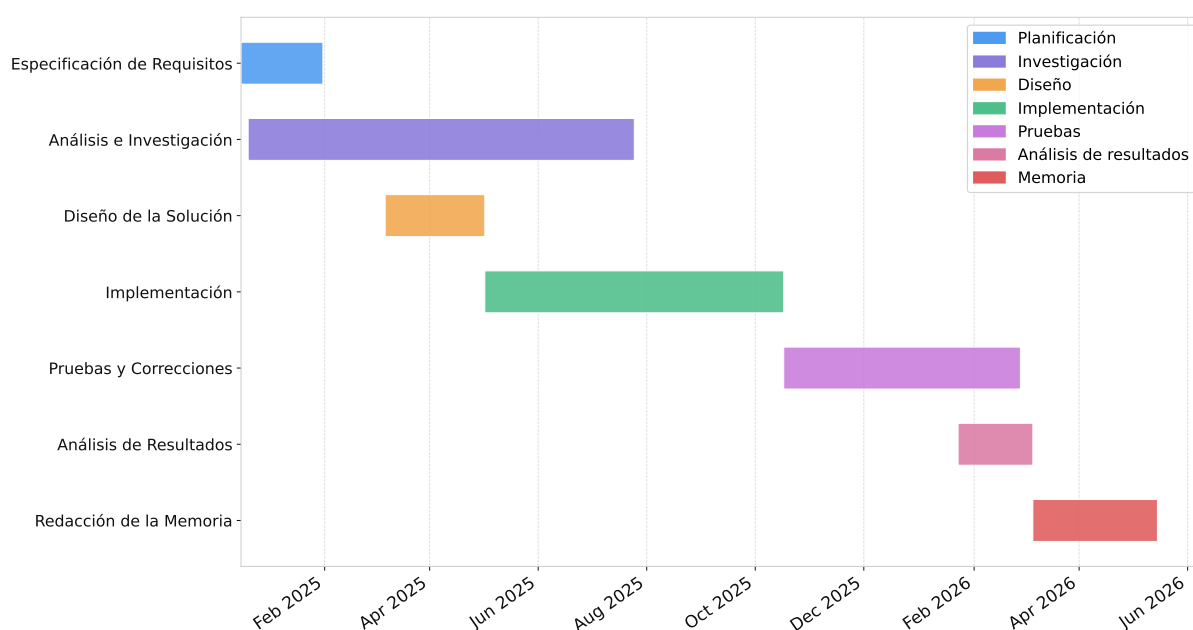


Figura 4.2: Diagrama de Gantt representando la duración de las fases.

Fuente: Elaboración propia.

En el grado cursado, el TFG equivale 12 créditos ECTS, lo que representa a 300 horas de trabajo. Desde enero de 2025 hasta septiembre de ese mismo año se invirtieron unas 6 horas semanales, pues durante el cuatrimestre que cubre ese período se cursaban tres asignaturas. Durante la primera mitad del curso 2025-2026, que cubre de septiembre a diciembre, se redujo la dedicación a aproximadamente 3 horas semanales debido a las seis



asignaturas matriculadas. Por último, desde enero de 2026, una vez finalizado el período de exámenes, se pudieron invertir unas 10 horas semanales.

Teniendo en cuenta el tiempo invertido por semana, este TFG ha supuesto alrededor de 400 horas de trabajo. En la Figura 4.2 se muestran las diferentes fases del proyecto y el número de semanas que ha ocupado cada una.

Las tareas en las que más tiempo se ha invertido son el análisis e investigación, la implementación y las pruebas y correcciones. El análisis e investigación es una fase extensa y necesaria para entender, principalmente, los fundamentos de PSO. La implementación es el núcleo del proyecto y abarca un gran volumen de horas, no solo por trabajar con *frameworks* con los que no se habían trabajado anteriormente (como *NetworkX* o *PySwarms*), sino también por el constante refinamiento en cada iteración del desarrollo en espiral. Por último, las pruebas y correcciones requirieron una dedicación considerable, pues ayudan a garantizar la calidad de las soluciones y el correcto funcionamiento del sistema.

Capítulo 5

Descripción del Algoritmo Propuesto

En esta sección se presenta detalladamente la implementación del algoritmo desarrollado para abordar el objetivo de este trabajo, sus atributos y su evolución hasta llegar a la solución final.

5.1. Elección de la Técnica de Optimización

Como se ha visto en el apartado del estado del arte, los problemas de optimización se pueden abordar con diferentes técnicas dependiendo de las características del problema.

Una de las técnicas que ofrece soluciones óptimas, a costa de un gran tiempo de cómputo, es la programación lineal entera (ILP). El estudio *Networking for a Low-Carbon Future: An ILP-Based Carbon-Aware Approach for SDN* [6] muestra cómo esta técnica es capaz de reducir las emisiones de carbono en hasta un 84.5%, aunque el tiempo de ejecución llegó a alcanzar los 17 minutos, lo que es prohibitivo en escenarios tan cambiantes.

Como alternativa a soluciones basadas en ILP, se pueden usar técnicas de *Deep Learning* (DL). Pero estas técnicas, además de requerir una gran cantidad de datos para entrenar el modelo, no son viables para este tipo de problemas, pues en el momento en el que se modifique la topología de red, el modelo entrenado dejará de ser válido, lo que obligará a volver a entrenar el modelo desde cero, con la consecuente pérdida de tiempo

y recursos.

Los algoritmos genéticos pueden ser una buena opción, pero suelen requerir una considerable cantidad de tiempo de ejecución, especialmente para problemas con un gran espacio de búsqueda. Al estar tratando con un escenario en el que el tiempo es crítico, pues las necesidades de la red cambian a lo largo del día, las soluciones basadas en algoritmos genéticos no son factibles.

Por otro lado, los algoritmos voraces son rápidos y fáciles de implementar. Estas soluciones eliminan la problemática de los tiempos de ejecución largos que presentan las técnicas anteriores. Sin embargo, toman decisiones basadas únicamente en la información local sin considerar el impacto a largo plazo. Esto puede llevar a soluciones subóptimas o, incluso, a soluciones inviables si se toman decisiones que comprometen la conectividad de la red. Realmente, que un algoritmo voraz encuentre una solución subóptima no es algo inherentemente malo, pues ya se estaría hablando de una mejora significativa respecto a la situación inicial, pero el hecho de que pueda encontrar soluciones inviables es un riesgo que bajo ningún concepto se puede permitir.

En cambio, las soluciones basadas en PSO son una gran opción para este tipo de problemas, ya que son capaces de explorar el espacio de búsqueda de manera eficiente y encontrar soluciones, aunque no sean óptimas, en un tiempo razonable. Respecto a los algoritmos voraces, PSO ofrece la ventaja de ofrecer siempre soluciones factibles. Además, como PSO es un algoritmo de optimización global, tiene la capacidad de evitar caer en óptimos locales. Teniendo en cuenta que se tratan de problemas con grandes espacios de búsqueda, es fundamental evitar a toda costa los óptimos locales.

Otra ventaja de PSO es que existen numerosas implementaciones disponibles, lo que facilita su uso y permite centrarse en la modelización de la función objetivo. Además, PSO es un algoritmo relativamente sencillo de ajustar, con pocos parámetros a configurar, facilita la realización de barridos para encontrar la configuración que mejor se adapte al problema en cuestión.

Es por esto que se ha optado por usar PSO como técnica de optimización para abordar el problema de optimización de las emisiones de CO_2 en redes cableadas.

5.2. *Particle Swarm Optimization*

Como en la mayoría de algoritmos de optimización evolutivos, PSO se basa en la idea de iterar sobre un conjunto de soluciones candidatas, partículas, y actualizar estas soluciones en función de su rendimiento.

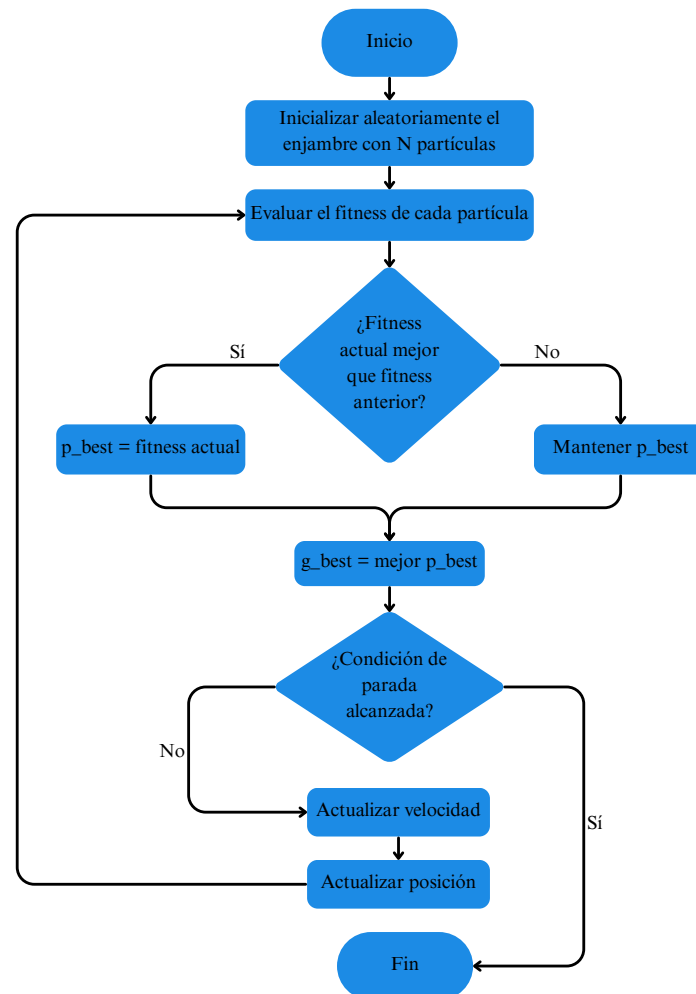


Figura 5.1: Diagrama de flujo de PSO.
Fuente: Elaboración propia.

Para entender mejor el diagrama de flujo que se muestra en la Figura 5.1, se va a usar un sencillo ejemplo: personas buscando una localización en un mapa.

1. PSO inicializa aleatoriamente el enjambre con un número determinado de partículas, que traducido al ejemplo, sería posicionar a las personas en el mapa de manera



aleatoria.

2. Luego, cada persona evalúa la calidad de su posición (cada partícula evalúa el fitness), es decir, cómo de cerca está de la localización objetivo.
3. A continuación, cada persona compara su posición actual con la mejor posición que ha encontrado hasta ese momento (p_best) y, en caso de ser superior, actualiza su mejor posición personal.
4. Después, los individuos se comunican entre sí para compartir información sobre sus mejores posiciones personales, lo que les permite conocer la mejor posición global encontrada por el grupo (g_best).
5. Si se ha alcanzado la condición de parada, como puede ser un número máximo de intentos, el grupo comunica la mejor posición global encontrada, que sería la localización más cercana al objetivo. Si no se ha alcanzado la condición de parada, cada persona, en función de la información recabada hasta el momento, se mueve a otra posición (actualizar velocidad y actualizar posición) y se vuelve a evaluar la calidad de su nueva posición, empezando así una nueva iteración.

5.3. Parámetros de PSO

En esta sección se detallan los parámetros que influyen en la ejecución de PSO, como pueden ser las partículas, el coeficiente de inercia, el coeficiente cognitivo, el coeficiente social, el número de vecinos y las dimensiones del espacio de búsqueda. Cada uno de estos parámetros tiene un impacto significativo en el rendimiento del algoritmo y en la calidad de las soluciones encontradas, por lo que es importante entender su función y cómo afectan al comportamiento del algoritmo.

5.3.1. Partícula

El concepto de partícula ya se ha comentado brevemente con anterioridad, y es que una partícula es un agente individual que representa una solución candidata al problema



de optimización. Como se ha comentado en el ejemplo anterior, se puede entender por partícula a una persona buscando la ubicación objetivo del mapa. De hecho, este ejemplo no se aleja mucho de la inspiración original de PSO, puesto que se basa en el comportamiento social de bandadas de pájaros o bancos de peces moviéndose en grupo hacia un objetivo común.

5.3.2. Función objetivo

La función objetivo es una parte fundamental de cualquier algoritmo de optimización, pues se trata de la función que se usa para evaluar cada solución candidata y determinar qué tan buena es esa solución. En el caso de PSO, la función objetivo se utiliza para evaluar la calidad de cada partícula en cada iteración, y se utiliza para actualizar la mejor posición personal de cada partícula y la mejor posición global del enjambre.

5.3.3. Velocidad

En PSO, las partículas tienen dos atributos principales, siendo la velocidad uno de ellos. Esta determina en qué dirección y cuánto se mueve cada partícula a lo largo del espacio de búsqueda. La velocidad se actualiza en cada iteración en función de la mejor posición personal de la partícula y la mejor posición global del enjambre, lo que permite que las partículas exploren el espacio de búsqueda de manera eficiente. Una velocidad alta puede permitir que las partículas exploren el espacio de búsqueda de manera más amplia, pero también puede hacer que el algoritmo sea más propenso a saltar sobre buenas soluciones. Por otro lado, una velocidad baja puede hacer que el algoritmo sea más lento, pero también puede ayudar a que las partículas se concentren en áreas prometedoras del espacio de búsqueda.

5.3.4. Posición

El segundo atributo principal de las partículas es la posición, que representa una posible solución al problema de optimización. La diferencia entre la posición y la partícula,



pues las definiciones pueden ser un poco confusas, es que la posición es el valor actual de la solución representada por la partícula, mientras que la partícula es el agente que se mueve a lo largo del espacio de búsqueda. Siguiendo con el ejemplo de las personas y el mapa, la posición sería el lugar específico donde se encuentra un individuo en un momento dado. La posición se actualiza en cada iteración en función de la velocidad de la partícula, lo que permite que las partículas se muevan a través del espacio de búsqueda y encuentren mejores soluciones.

5.3.5. Coeficiente de inercia

El coeficiente de inercia es un parámetro que controla la influencia de la velocidad anterior en la actualización de la velocidad actual de cada partícula. Un coeficiente de inercia alto hace que las partículas mantengan su velocidad anterior, lo que puede ayudar a que el algoritmo explore el espacio de búsqueda de manera más amplia. Por otro lado, un coeficiente de inercia bajo hace que las partículas sean más propensas a cambiar su velocidad en función de la mejor posición personal y global, lo que puede ayudar a que el algoritmo se concentre en áreas prometedoras. Es importante encontrar un valor adecuado para este parámetro en función del problema específico, ya que un valor demasiado alto o demasiado bajo puede afectar negativamente al rendimiento del algoritmo.

5.3.6. Coeficiente cognitivo

El coeficiente cognitivo es un parámetro que controla la influencia de la mejor posición personal de cada partícula en la actualización de su velocidad. Un coeficiente cognitivo alto hace que las partículas se centren más en su propia experiencia, lo que ayuda a que el algoritmo se centre en áreas prometedoras. Por otro lado, un coeficiente cognitivo bajo hace que las partículas sean más propensas a seguir la mejor posición global del enjambre, lo que puede ayudar a que el algoritmo explore el espacio de búsqueda más ampliamente.



5.3.7. Coeficiente social

El coeficiente social es un parámetro que controla la influencia de la mejor posición global del enjambre en la actualización de la velocidad de cada partícula. Un coeficiente social alto hace que las partículas se centren más en la experiencia colectiva del enjambre, lo que puede ayudar a que el algoritmo explore el espacio de búsqueda más ampliamente. Por otro lado, un coeficiente social bajo hace que las partículas sean más propensas a centrarse en su propia experiencia, así el algoritmo se concentra en áreas prometedoras. Es importante encontrar un equilibrio adecuado entre el coeficiente cognitivo y social, ya que un desequilibrio puede desembocar en soluciones subóptimas.

5.3.8. Número de vecinos

Este parámetro controla la cantidad de partículas que se consideran como vecinos de cada partícula en la actualización de su velocidad. Si el número de vecinos es alto, cada partícula se ve influenciada por un gran número de otras partículas, lo cual hace que se centre más en la experiencia colectiva del enjambre. En cambio, si el número de vecinos es bajo, cada partícula se ve influenciada por un número reducido de otras partículas, centrándose más en su propia experiencia.

5.3.9. Dimensiones del espacio de búsqueda

Este parámetro define el número de dimensiones que tiene el espacio de búsqueda, lo que a su vez determina la cantidad de variables que se están optimizando. Por lo general, cuantas más dimensiones tenga el espacio de búsqueda, más complejo será el problema de optimización, ya que habrá más variables a considerar, pero puede permitir encontrar mejores soluciones. Por el contrario, un espacio de búsqueda con pocas dimensiones puede ser más fácil de explorar, pero también puede limitar la calidad de las soluciones encontradas.



5.4. Aproximación al Problema

Con los conceptos básicos de PSO ya explicados, es importante entender cómo se ha aplicado este algoritmo al problema específico de optimización de las emisiones de CO_2 en redes SDN.

La primera solución que se planteó fue transformar el espacio discreto del problema en un espacio continuo, lo que permitiría usar PSO en su forma original. Sin embargo, esta aproximación terminó descartándose, pues la umbralización hacía que PSO trabajase a ciegas. Es decir, no era capaz de discernir entre los cambios que se producían por debajo del umbral, lo que se considera como un enlace apagado, por tanto PSO no sabía si esos cambios apuntaban a una mejora o a un empeoramiento de la solución, lo que generaba un comportamiento errático del algoritmo.

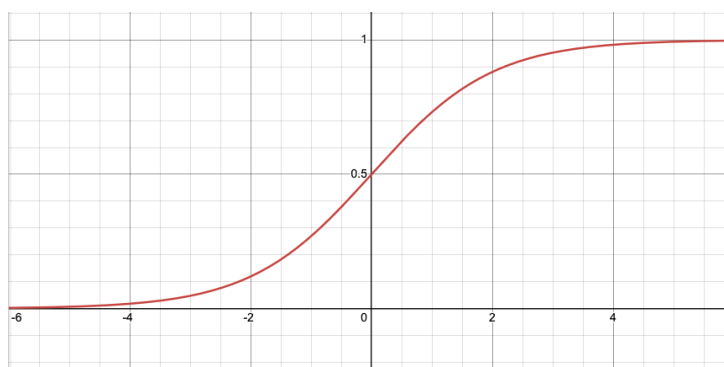


Figura 5.2: Representación gráfica de la función sigmoide.

Fuente: Elaboración propia.

Más adelante, se iteró sobre la idea anterior de transformar el espacio discreto en continuo, pero esta vez usando una función sigmoide para umbralizar a través del cálculo de la probabilidad de que un enlace esté activo o apagado. Sin embargo, y como se muestra en la Figura 5.2, se observó que, a medida que el valor continuo de la partícula, x , se aleja del cero, la curva se vuelve casi plana. Esto hace que, si el algoritmo decide que el enlace debe estar encendido, la velocidad de PSO empuje el valor de x hacia números positivos cada vez más altos. Pero si más tarde el algoritmo decide que ese enlace en realidad debe estar apagado, PSO tardará muchas iteraciones en generar una velocidad lo suficientemente negativa como para arrastrar el valor de x de vuelta al cero. En todo



ese tiempo, la partícula está estancada y pierde capacidad de exploración.

Finalmente, se optó por hacer uso de adaptaciones de PSO para espacios de búsqueda discretos. De esta manera, se evitan los problemas de "cagera" de la umbralización y las pérdidas de capacidad de exploración que se observaban con la función sigmoide. Además, esto permite que PSO trabaje directamente con el espacio discreto del problema, mejorando así la eficiencia del algoritmo.

En el entorno de las redes SDN, la solución propuesta se implementaría en el controlador SDN, lo que permitiría a PSO tener acceso a información en tiempo real sobre el estado de la red y evaluar a lo largo del día los cambios que habría que hacer en cada momento para mantener las emisiones de CO_2 al mínimo.

5.4.1. Estrategias de manejo de restricciones

Para que el algoritmo sea realmente efectivo, se han de tener en cuenta las restricciones estrictas de la red, como pueden ser asegurar la conectividad total o no sobrepasar el ancho de banda. Para ello, se han evaluado dos aproximaciones de la función objetivo: PSO con penalización estricta y PSO con manejo de violación de restricciones.

PSO con penalización estricta

En esta variante, cuando una partícula propone una configuración de red que viola cualquiera de las restricciones, la función objetivo devuelve el valor infinito. En otras palabras, la posición de la partícula sufre una "muerte súbita", pues corta de golpe en el momento en el que no cumple una restricción.

La ventaja principal que presenta esta estrategia es la rapidez, pues el algoritmo descarta de manera inmediata una solución que no cumple con alguna restricción. Pero esta rapidez computacional le genera "cagera".^a PSO al no ser capaz de saber cuán buena es una solución inválida.

PSO con manejo de violación de restricciones

A diferencia de la variante anterior, el método *Violation Constraint Handling* (VCH) no descarta una solución de manera inmediata, sino que contabiliza la cantidad de veces que se ha violado alguna restricción (v) y la multiplica por un factor de penalización (P). Este valor se suma a las emisiones de CO_2 :

$$f_{fitness} = f_{emisiones} + v \cdot P \quad (5.1)$$

Este manejo más controlado de las violaciones proporciona a PSO un gradiente de información. De esta manera se puede discernir si una solución es menos mala que otra, lo que permite a las partículas atravesar zonas de inviabilidad para alcanzar regiones óptimas. Aunque es más lento que la versión anterior, puesto que el algoritmo debe realizar todos los cálculos para cada posición de cada partícula.

5.5. Implementación y Desarrollo del Algoritmo

En esta sección se usan las explicaciones realizadas en la sección anterior para definir el algoritmo que finalmente se ha desarrollado. Se detallan las decisiones tomadas durante el proceso de implementación, los desafíos encontrados y cómo se han abordado.

5.5.1. Configuración de las partículas

Como se ha comentado con anterioridad, una partícula representa una solución candidata al problema que se está tratando de optimizar. Siguiendo el ejemplo de las personas de la Sección 5.2, cada individuo representa una partícula evaluando las diferentes posiciones. En el caso de la optimización de las emisiones de CO_2 en redes SDN, cada partícula representa un agente que evalúa las diferentes posibles configuraciones de la red, es decir, diferentes combinaciones de enlaces encendidos y apagados.

Las partículas representan uno de los parámetros más importantes de PSO, pues su número y su configuración pueden afectar significativamente al rendimiento del algoritmo.

En este caso, se han configurado las partículas como vectores binarios, donde cada elemento del vector representa el estado de un enlace en la red, siendo 1 para un enlace encendido y 0 para un enlace apagado. En la Figura 5.3 se muestra una sencilla representación de una red como partícula de PSO.

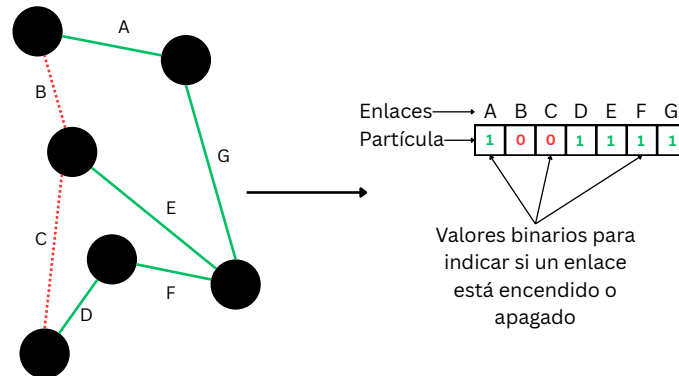


Figura 5.3: Representación de una red sencilla como partícula de PSO.
Fuente: Elaboración propia.

La cantidad de partículas se ha establecido buscando un equilibrio entre la calidad de las soluciones encontradas y el tiempo de ejecución del algoritmo. Se han realizado varios barridos con la matriz de tráfico más cargada de la red Abilene, TM5, para averiguar cuál es el número más adecuado de partículas. Además, se aprovechó este mismo barrido para saber qué configuración de $c1$ y $c2$, coeficientes cognitivo y social, respectivamente, es la más eficiente. Los resultados que se obtuvieron en un primer momento son los que se pueden observar en la Figura 5.4. Apenas hay que analizar la gráfica para darse cuenta de que los resultados son erráticos y poco concluyentes.

De manera intuitiva, se podría pensar que a medida que se incrementa el número de partículas, la calidad de las soluciones encontradas debería mejorar, ya que el algoritmo tiene más agentes explorando el espacio de búsqueda. Sin embargo, como puede observarse en la gráfica, no se aprecia una tendencia clara, sino que los resultados mejoran y empeoran a medida que se incrementa la cantidad de partículas.

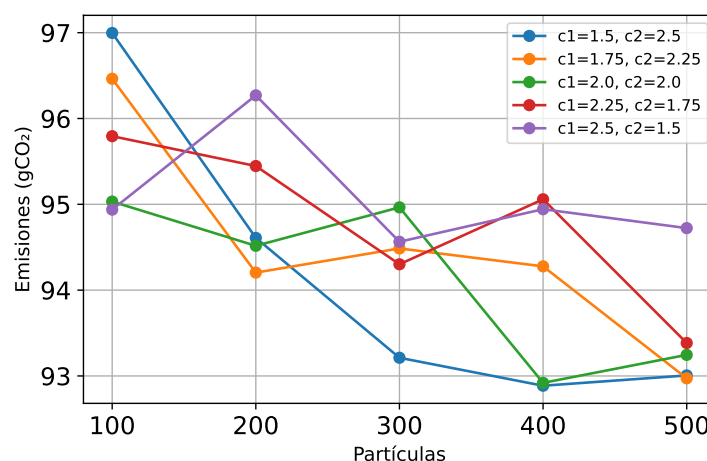


Figura 5.4: Resultados del barrido de partículas para la TM5 de Abilene. Iteraciones fijas en 1500.

Fuente: Elaboración propia.

Tras estos primeros resultados, se decidió investigar el por qué de los mismos. Tras la realización de diferentes pruebas, las cuales se comentarán con detalle más adelante en el Capítulo 6, se descubrió un factor que estaba afectando a los resultados de manera significativa: la aleatoriedad con la que se inician las partículas. Dado que PSO es un algoritmo estocástico, la inicialización aleatoria de las partículas puede llevar a resultados muy diferentes en cada ejecución, lo que explica la falta de una tendencia clara en los resultados del barrido. Para abordar este problema, se decidió inicializar las partículas de dos maneras: una de las partículas del enjambre se inicializa con todos los enlaces encendidos y el resto de partículas se inicializan con un porcentaje de enlaces encendidos que se comprende entre el 50 % y el 90 %.

En la Figura 5.5 se pueden observar los resultados del barrido de partículas con esta nueva estrategia de inicialización. En la gráfica se aprecia una clara tendencia a mejorar los resultados a medida que se incrementa el número de partículas, lo que confirma las sospechas sobre la influencia de la aleatoriedad en los resultados anteriores. Aunque todavía hay algún que otro pico, estos son despreciables en comparación con la mejora obtenida, pues disminuyen las emisiones entre 8 y 10 gCO_2 .

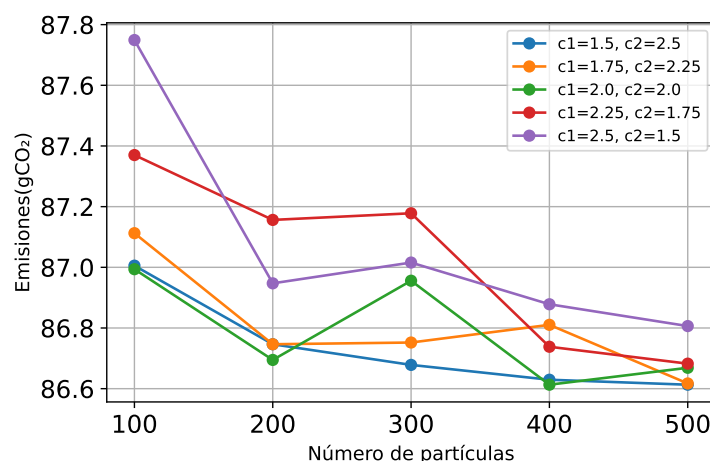


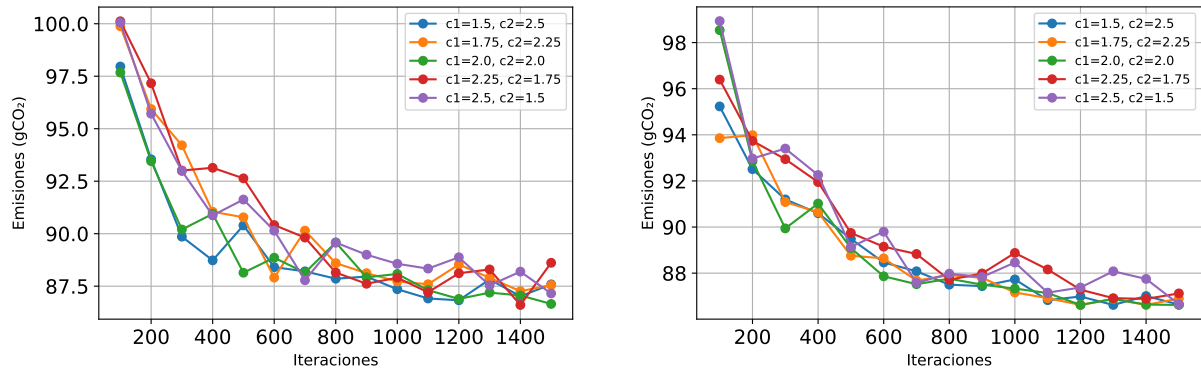
Figura 5.5: Resultados del barrido de partículas para la TM5 de Abilene con inicialización controlada de partículas. Iteraciones fijas en 1500.

Fuente: Elaboración propia.

Finalmente, se decidió usar 100 y 200 partículas para las pruebas finales. Por un lado, a partir de 200 partículas, las mejoras obtenidas no justificaban el incremento en el tiempo de ejecución. Por otro lado, aunque los resultados obtenidos con 100 partículas son ligeramente peores, la diferencia en el tiempo de ejecución es lo suficientemente alta como para justificar el uso de 100 partículas.

5.5.2. Iteraciones

Otro de los parámetros clave de PSO es el número de iteraciones, que determina cuántas veces las partículas tendrán que relizar el proceso de evaluación y actualización de sus posiciones. En general, un mayor número de iteraciones puede permitir que el algoritmo explore el espacio de búsqueda de manera más exhaustiva y encontrar mejores soluciones, pero también puede aumentar significativamente el tiempo de ejecución. Análogamente a lo que se hizo con las partículas, se realizaron dos barridos sobre la matriz TM5 de Abilene para averiguar cuál es el número de iteraciones más adecuado. Ambos barridos se realizaron desde 100 iteraciones hasta 1500, incrementando de 100 en 100, dejando fijas las partículas en 100 y 200, además de usar la misma estrategia de inicialización. También se aprovecharon estos barridos para averiguar qué configuración de $c1$ y $c2$ es la más eficiente.



(a) Barrido con partículas fijas en 100.
Fuente: Elaboración propia.

(b) Barrido con partículas fijas en 200.
Fuente: Elaboración propia.

Figura 5.6: Resultados de los barridos de iteraciones para la TM5 de Abilene con 100 y 200 partículas.

En los barridos mostrados en la Figura 5.6 se aprecia una clara tendencia a mejorar los resultados a medida que aumentan las iteraciones. En ambos casos, se observa que a partir de 600 iteraciones las mejoras que se obtienen son casi despreciables, por lo que se decidió fijar en 600 las iteraciones para las pruebas finales. De todas maneras, se decidió usar también 1200 iteraciones para comprobar si esta segunda configuración obtenía mejores resultados en escenarios más complejos, como pueden ser redes más grandes y con más tráfico.

Estas gráficas aportan información muy interesante sobre la influencia de las partículas en los resultados. En la Figura 5.6a, aunque todas las configuraciones de $c1$ y $c2$ mejoran los resultados al aumentar las iteraciones, se observa cierta disparidad entre las diferentes configuraciones. En cambio, en la Figura 5.6b, hay una mayor convergencia en los resultados obtenidos con las diferentes combinaciones de $c1$ y $c2$, lo que refuerza la decisión de usar 100 y 200 partículas para las pruebas finales.

5.5.3. Coeficientes cognitivo y social

Para determinar la configuración más adecuada de los coeficientes cognitivo y social, se aprovecharon los barridos realizados en las Subsecciones 5.5.1 y 5.5.2. En todos los barridos realizados, las configuraciones que ofrecían los mejores resultados eran aquellas

en las que el coeficiente social era mayor que el cognitivo, es decir, aquellas en las que las partículas se centraban más en la explotación que en la exploración. Concretamente, las dos mejores configuraciones era $c_1 = 1,5$ y $c_2 = 2,5$ (gráfica azul) y $c_1 = 1,75$ y $c_2 = 2,25$ (gráfica naranja). Puesto que desde un principio se quería buscar que el algoritmo se centrara más en la explotación, se terminó optando por la gráfica naranja, es decir, por la configuración $c_1 = 1,75$ y $c_2 = 2,25$, pues ofrece un buen equilibrio entre la exploración personal y la explotación colectiva, dando cierta prioridad a esta última.

5.5.4. Inercia, vecinos y dimensiones del espacio de búsqueda

Un valor coherente para que el coeficiente de inercia, representado por w , fomente que las partículas se centren más en la explotación que en la exploración, pero sin dejar de lado esta última, es 0.7. Lo que indica este valor es que para la siguiente iteración, a la hora de actualizar la velocidad, se tendrá en cuenta en un 70 % la velocidad de la iteración anterior. Esto permite a las partículas explotar los resultados grupales a buen ritmo pero deja margen de maniobra en caso de que estos resultados no terminen de ser fructíferos.

En el caso del número de vecinos, la cantidad fijada para todas las pruebas ha sido 100. Esto permite tener dos escenarios durante los experimentos: cuando el enjambre se componga de 100 partículas, cada una tendrá acceso a la información de todo el enjambre; cuando sea de 200, cada agente solo considerará la mitad. Estos dos escenarios son muy interesantes, ya que el primero favorece una rápida convergencia hacia la solución global, mientras que el segundo fomenta más la exploración.

Por último, las dimensiones del espacio de búsqueda dependen por completo de la topología de red, puesto que es el número de enlaces unidireccionales de la red lo que las define. Esto se debe a que cada partícula representa una solución candidata en la que se asigna un estado a cada enlace, por lo que el número de variables de dicha solución coincide directamente con el número de enlaces de la red. Por lo tanto, redes con mayor número de enlaces darán lugar a espacios de búsqueda más grandes, incrementando así la complejidad del problema.

5.5.5. Función objetivo y pseudocódigo

Para modelar la función que usará el algoritmo de PSO para evaluar la calidad de las posibles soluciones, se parte de la fórmula propuesta en el artículo *Exploring the Benefits of Carbon-Aware Routing* [2]:

$$C_{tot}(G, X, \lambda, c, \beta) = \sum_{n=1}^V (x_n * \lambda_n * c_n) + \sum_{l=1}^E \beta_l (c_{l_{n_1}} + c_{l_{n_2}}) \quad (5.2)$$

Donde:

- G es la topología de la red, representada como un grafo.
- V y E son el número de nodos y enlaces de la red, respectivamente.
- x_n es la intensidad de flujo en el nodo n .
- c_n es la tasa de emisiones de CO_2 asociada al nodo n .
- λ_n es la potencia dinámica incremental por tasa de datos del nodo n .
- β_l es el incremento del consumo de energía al habilitar un puerto.
- $c_{l_{n_1}}$ y $c_{l_{n_2}}$ son las emisiones de CO_2 asociadas a los nodos n_1 y n_2 que están conectados por el enlace l .

El primer sumatorio representa el consumo dinámico de energía asociado al tráfico que pasa por cada nodo, es decir, cuanto más tráfico pase por un nodo, más energía consumirá ese nodo. El segundo sumatorio se corresponde con el consumo estático de energía asociado a los enlaces que están activos. Un enlace activo implica que hay un puerto habilitado en cada uno de los nodos que conecta, lo que genera un consumo de energía adicional. La Figura 5.7 representa gráficamente el modelo de consumo de energía de un router, lo cual refuerza la explicación anterior: la potencia estática es el consumo base de energía de un router y no se puede reducir. La potencia de los puertos es la cantidad de energía extra que se consume al habilitar un puerto. Por último, la componente azul del modelo es la potencia dinámica, que depende de la carga de tráfico del router.

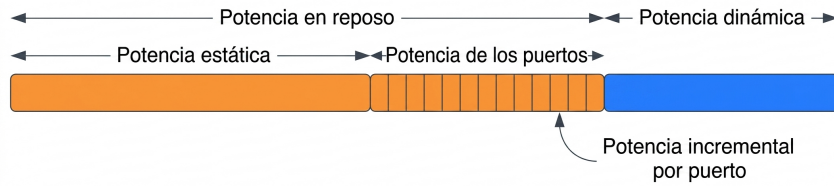


Figura 5.7: Modelo de consumo de energía de un router.

Fuente: José Antonio Gómez de la Hiz [6].

A lo largo de este trabajo, se ha mencionado en varias ocasiones la importancia de no dejar sin conexión ninguna parte de la red. Si a eso se le junta querer reducir la carga de nodos y enlaces en zonas de altas emisiones, se genera la necesidad de buscar los caminos que cumplan ambos requisitos, necesidad que puede satisfacerse con la estrategia KSP. Esta técnica permite encontrar los K caminos más cortos entre dos puntos, aunque en este caso el criterio de eficiencia no es la distancia sino las emisiones de carbono. Para ello, se asigna a cada arista del grafo que representa la red su intensidad de carbono correspondiente, de manera que KSP calcule las rutas ambientalmente más eficientes. Así, si la ruta que menos emisiones genera no tiene capacidad de enrutar todo el tráfico, se evaluaría la siguiente ruta.

El Algoritmo 1 presenta el pseudocódigo de la versión para PSO normal. Dada una matriz de tráfico, se asigna cada flujo al mejor camino disponible, procesando primero las demandas más grandes. Un camino se considera válido si solamente utiliza enlaces activos de la red y dispone de capacidad suficiente para satisfacer la demanda. Si algún flujo no puede ser asignado, la solución se considera inválida y se devuelve $+\infty$. Una vez asignados todos los flujos, se calculan las emisiones de carbono: la potencia dinámica P_{dyn} , proporcional al tráfico en cada nodo, sumada a la potencia de los puertos P_{ports} , asociada a los enlaces activos de la topología.

Algorithm 1 Cálculo de la emisiones totales de la red para PSO normal

Require: position[N][N], flow_matrix[N][N], all_k_paths, nodes_max_flow[N][N], carbon_intensity[N]

Ensure: Emisiones totales de carbono de la red

```

1: Inicializar nodes_traffic  $\leftarrow 0$ , links_traffic  $\leftarrow 0$ ,  $P_{\text{dyn}} \leftarrow 0$ ,  $P_{\text{ports}} \leftarrow 0$ 
2:  $\mathcal{A} \leftarrow \{(i, j) \mid \text{position}[i][j] = 1\}$ 
3: Ordenar demandas  $\mathcal{D}$  de forma descendente por flow_matrix[ $s$ ][ $d$ ]
4: for  $(s, d, \delta) \in \mathcal{D}$  do
5:    $p^* \leftarrow$  primer camino en all_k_paths[ $(s, d)$ ] válido y con capacidad  $\geq \delta$ 
6:   if  $p^* = \text{NONE}$  then return  $+\infty$ 
7:   end if
8:   Actualizar links_traffic y nodes_traffic según  $p^*$  y  $\delta$ 
9: end for
10: for  $x \leftarrow 0$  to  $N - 1$  do
11:    $c_x \leftarrow \text{carbon\_intensity}[x]/1000$ 
12:    $P_{\text{dyn}} += \text{nodes\_traffic}[x] \cdot \lambda_n \cdot c_x$ 
13:   for  $y \neq x$  con position[ $x$ ][ $y$ ]  $\neq 0$  do
14:      $P_{\text{ports}} += \beta_l \cdot (c_x + c_y)$ 
15:   end for
16: end for
17: return  $P_{\text{dyn}} + P_{\text{ports}}$ 

```

El Algoritmo 2 presenta la variante que introduce el mecanismo de penalización VCH. Cuando un flujo no puede ser asignado por ausencia de un camino válido o falta de capacidad, esta versión lo cuenta como una violación. Al finalizar la asignación, el número de violaciones se multiplica por un factor de penalización M y se añade a las emisiones de carbono. Esto permite que el algoritmo siempre devuelva un valor finito, penalizando las soluciones que incumplen las restricciones de la red. En el Algoritmo2, el cambio puede observarse en las líneas...

Algorithm 2 Cálculo de la emisiones totales de la red para PSO VCH

Require: position[N][N], flow_matrix[N][N], all_k_paths, nodes_max_flow[N][N], carbon_intensity[N], M

Ensure: Intensidad de carbono total de la red, penalizada por violaciones

```

1: Inicializar nodes_traffic  $\leftarrow 0$ , links_traffic  $\leftarrow 0$ ,  $P_{\text{dyn}} \leftarrow 0$ ,  $P_{\text{ports}} \leftarrow 0$ ,  $v \leftarrow 0$ 
2:  $\mathcal{A} \leftarrow \{(i, j) \mid \text{position}[i][j] = 1\}$ 
3: Ordenar demandas  $\mathcal{D}$  de forma descendente por flow_matrix[ $s$ ][ $d$ ]
4: for  $(s, d, \delta) \in \mathcal{D}$  do
5:    $p^* \leftarrow$  primer camino en all_k_paths[ $(s, d)$ ] válido y con capacidad  $\geq \delta$ 
6:   if  $p^* = \text{NONE}$  then
7:      $v += 1$   $\triangleright$  Contabilizar violación y continuar
8:   else
9:     Actualizar links_traffic y nodes_traffic según  $p^*$  y  $\delta$ 
10:  end if
11: end for
12: for  $x \leftarrow 0$  to  $N - 1$  do
13:    $c_x \leftarrow \text{carbon\_intensity}[x]/1000$ 
14:    $P_{\text{dyn}} += \text{nodes\_traffic}[x] \cdot \lambda_n \cdot c_x$ 
15:   for  $y \neq x$  con position[ $x$ ][ $y$ ]  $\neq 0$  do
16:      $P_{\text{ports}} += \beta_l \cdot (c_x + c_y)$ 
17:   end for
18: end for
19: return  $P_{\text{dyn}} + P_{\text{ports}} + v \cdot M$ 

```

Capítulo 6

Pruebas y Resultados

A lo largo de este capítulo se presentan las pruebas realizadas para evaluar el rendimiento del algoritmo, así como los resultados obtenidos y su análisis. Además, se detalla la configuración final del algoritmo, donde se explican las librerías utilizadas y otros aspectos relevantes como las especificaciones del ordenador en el que se han realizado las pruebas.

6.1. Detalles de la Implementación

Para la implementación del algoritmo desarrollado, se ha hecho uso del lenguaje de programación Python, debido a su versatilidad y a la gran cantidad de librerías disponibles para el desarrollo de algoritmos de optimización y análisis de datos.

Inicialmente se valoraron dos herramientas para el desarrollo del algoritmo: Visual Studio Code y PyCharm. Finalmente se optó por PyCharm, puesto que ofrece diferentes funcionalidades que facilitan en gran medida el desarrollo de código, como pueden ser la integración de sistemas de control de versiones, la depuración de código o la gestión de entornos virtuales.

En cuanto a las librerías, se han utilizado principalmente las siguientes:

- NumPy [9]: para el manejo de arrays y matrices, así como la carga de datos desde archivos de texto plano y CSV.



- NetworkX [10]: para la creación y manipulación de grafos, fundamental para poder modelar las topologías de red usadas.
- Matplotlib [11]: para la visualización de resultados y gráficos.
- PySwarms [12]: para el uso de algoritmos de PSO en espacios discretos.
- SciPy [13]: para cálculos estadísticos adicionales, como los intervalos de confianza.

PySwarms es una librería de Python de código abierto diseñada para la implementación de algoritmos de PSO. Ofrece diferentes opciones de control y configuración sobre estos, sus parámetros, las funciones de coste y las condiciones de convergencia. También dispone de un módulo discreto que permite realizar optimizaciones en espacios de búsqueda discretos, especialmente útil en escenarios donde las variables de decisión toman valores discretos o enteros.

Una de las opciones de configuración es la paralelización para acelerar el proceso de evaluación de las partículas, lo que ha sido de gran ayuda para reducir el tiempo de ejecución del algoritmo. Aunque la CPU presente *hyper-threading*, para tareas de cómputo intensivo Python solo es capaz de ejecutar en paralelo de forma efectiva tantos procesos como núcleos físicos tenga el procesador, es decir, si el ordenador tiene una CPU de 6 núcleos y 12 hilos, por mucho que se hayan creado 40 procesos, solamente 6 se ejecutan a la vez. Aún así, la paralelización de PySwarms ha permitido aprovechar al máximo la capacidad de la CPU y reducir significativamente los tiempos de ejecución.

Todas las pruebas y experimentos realizados se han llevado a cabo en un ordenador con las siguientes especificaciones:

- CPU: AMD Ryzen 5 3600x (6 núcleos, 12 hilos, frecuencia base de 3.8 GHz y frecuencia turbo de 4.4 GHz).
- GPU: AMD Radeon RX 7900 GRE (16GB GDDR6)
- RAM: 32GB DDR4 a 3200MT/s CL16



6.2. Topologías Utilizadas

Las pruebas se han realizado sobre diferentes topologías de red reales, cuya información se ha obtenido de la web SNDlib [14], las cuales se detallan a continuación:

- Abilene. La red más pequeña de las cuatro, usada en el Capítulo 5 para realizar los barridos de partículas e iteraciones.
- Nobel-Germany. Red de tamaño medio, se ha usado para evaluar el rendimiento del algoritmo en escenarios más complejos que el de Abilene.
- Geant. Esta red, que es ligeramente más grande que Nobel, se ha utilizado para comprobar que la solución sigue siendo viable en redes de un tamaño parecido al de Nobel.
- Germany-50. Al duplicar el tamaño de la red con respecto las dos anteriores, se obtiene un escenario con una complejidad lo suficientemente alta como para evaluar la escalabilidad del algoritmo y su rendimiento en redes más grandes.

Estas topologías se han seleccionado con el objetivo de cubrir diferentes tamaños de red y localización, lo que permite evaluar de forma contundente el rendimiento del algoritmo y su viabilidad. En la Tabla 6.1 se recogen las principales características de cada red.

Red	Nodos	Enlaces (unidir.)
Abilene	12	30
Nobel-Germany	17	52
Geant	22	72
Germany-50	50	176

Tabla 6.1: Topologías de red utilizadas en las pruebas.

Fuente: SNDlib.

Inicialmente, para las emisiones de carbono se iba a hacer uso de la API de Electricity Maps [15], pero debido a un cambio en la política de recuperación de datos por parte de los desarrolladores de la API, las zonas accesibles con un token de autenticación quedaron limitadas a una. Intentar hacer uso de sus *endpoints* significa definir un token por cada



zona que se quisiera acceder y desarrollar la lógica necesaria para traducir las coordenadas geográficas de los nodos a zonas del mapa de Electricity Maps para, posteriormente, buscar qué token tiene acceso a esa zona y realizar la petición a la API. Debido al aumento de complejidad en la lógica del código, se terminó optando por usar datos históricos de intensidad de carbono publicados en Electricity Maps.

6.3. Definición de las Pruebas

En esta sección, se definen diferentes pruebas que evaluarán el rendimiento del algoritmo. En estos experimentos se ejecuta el algoritmo con las diferentes configuraciones de parámetros y estrategias de manejo de restricciones obtenidas en el Capítulo 5. Para cada matriz de tráfico, se ha ejecutado el algoritmo de dos maneras: 20 ejecuciones y 100 ejecuciones. La primera permite obtener una visión general del comportamiento del algoritmo rápidamente, además de emplear la distribución T de Student para calcular los intervalos de confianza, pues resulta ser más adecuada para muestras pequeñas. La segunda, 100 ejecuciones, proporciona resultados estadísticamente más precisos y con intervalos de confianza más ajustados, calculados usando la distribución normal estándar al tratarse de una muestra lo suficientemente grande.

6.4. Resultados para Abilene

A continuación, se muestran diferentes gráficas elaboradas a partir de los resultados obtenidos de las pruebas realizadas sobre Abilene. Esta topología es una infraestructura de red situada en Estados Unidos que interconecta universidades y centros de investigación. Como se muestra en la Figura 6.1, se compone de 12 nodos y 30 enlaces unidireccionales.



Figura 6.1: Topología de Abilene.
Fuente: SNDlib [14].

6.4.1. Emisiones de referencia

Previo a mostrar los resultados obtenidos de las pruebas realizadas, es importante conocer cuáles son las emisiones de la red con todos los enlaces encendidos para poder evaluar correctamente las mejoras alcanzadas. Para ello, se ha definido un escenario de referencia en el que la red permanece completamente activa y los flujos se enrutan siguiendo el criterio de camino más corto, sin aplicar ningún tipo de optimización energética. En la Figura 6.2 se puede observar que las emisiones de la red son mayores a medida que aumenta el tráfico, pues el grado de utilización de los enlaces es un factor que influye bastante. Un detalle a tener en cuenta es que las emisiones mostradas en la gráfica no comienzan en cero, esto se ha hecho para observar claramente las diferencias entre cada matriz de tráfico.

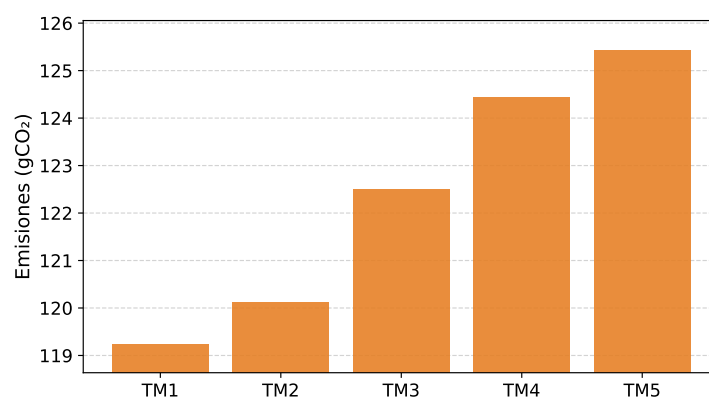


Figura 6.2: Emisiones de Abilene con todos los enlaces encendidos.
Fuente: Elaboración propia.

6.4.2. Emisiones después del apagado de enlaces

Las primeras pruebas que se realizaron con Abilene arrojaron unos resultados realmente prometedores. Haciendo uso de PSO normal con 200 partículas, 1200 iteraciones y 20 ejecuciones por cada matriz de tráfico, se han reducido las emisiones en hasta un 38.44 %. Además, como se muestra en la Figura 6.3, los intervalos de confianza son muy estrechos, lo que indica que con 20 ejecuciones el algoritmo presenta poca variabilidad y converge hacia soluciones de calidad.

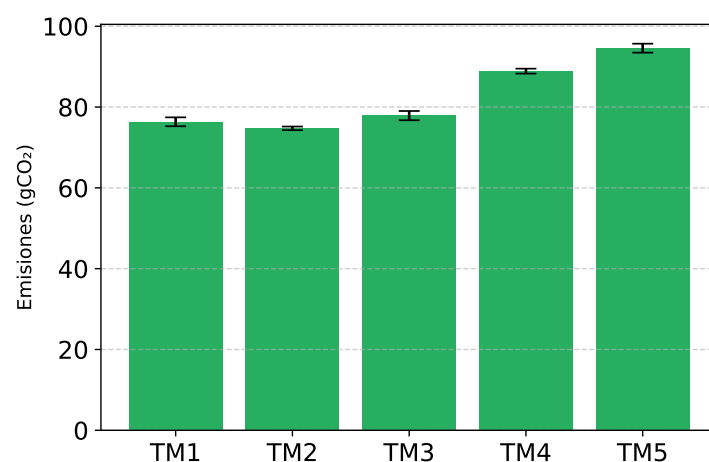


Figura 6.3: Emisiones de Abilene tras ejecutar PSO normal 20 veces por TM con 200 partículas y 1200 iteraciones. Intervalos de confianza al 95 %
Fuente: Elaboración propia.



En la Figura 6.3 se puede observar que el coste medio es ligeramente superior en TM1 que en TM2, lo que podría parecer contradictorio. Sin embargo, esta diferencia refleja una mayor dificultad del algoritmo para converger en el caso de TM1. De hecho, el mejor resultado individual obtenido para la TM1 (72.72) es inferior al de la TM2 (73.94), ambos con la misma configuración de enlaces, lo cual es coherente con el hecho de que una menor cantidad de tráfico se traduce en un menor coste total. El motivo por el cual el promedio en TM1 es mayor es que PSO encontró soluciones de peor calidad en varias ejecuciones debido a una mayor cantidad de óptimos locales al ser una matriz menos cargada. Esto aumenta la variabilidad en la calidad de las soluciones encontradas y, por tanto, eleva la media.

Al ejecutar el algoritmo 100 veces, los resultados que se muestran en la Figura 6.4 son prácticamente idénticos a los obtenidos en la prueba anterior, lo que confirma el buen funcionamiento del algoritmo. Los intervalos de confianza son aún más estrechos, lo cual es consecuencia del mayor tamaño muestral y refleja una estimación más precisa de la media. La consistencia entre los resultados de ambos experimentos evidencia que PSO converge de forma fiable hacia soluciones de calidad.

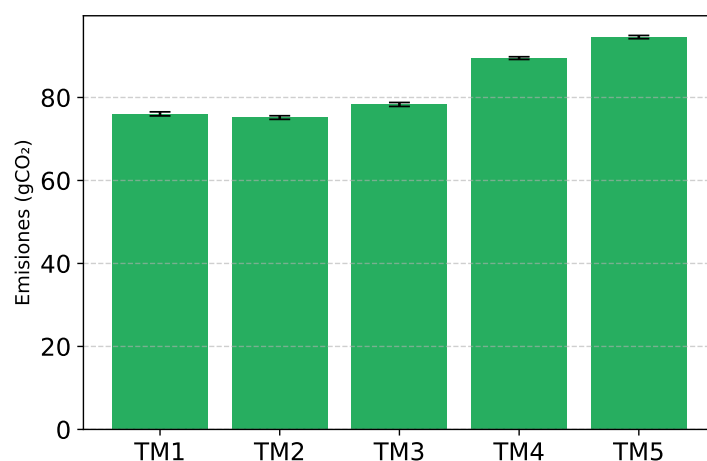


Figura 6.4: Emisiones de Abilene tras ejecutar PSO normal 100 veces por TM con 200 patículas y 1200 iteraciones. Intervalos de confianza al 95 %

Fuente: Elaboración propia.

Tras estas primeras pruebas, se repitieron los experimentos manteniendo la configuración pero esta vez usando PSO VCH. Los resultados usando esta segunda



estrategia son sorprendentes, pues se consigue una mejora de hasta un 20.80 % con respecto a PSO normal. En la Figura 6.5 se muestran estos incrementos de calidad en las soluciones, donde, además, se pueden observar unos intervalos de confianza muy estrechos, reforzando una vez más la estabilidad del algoritmo.

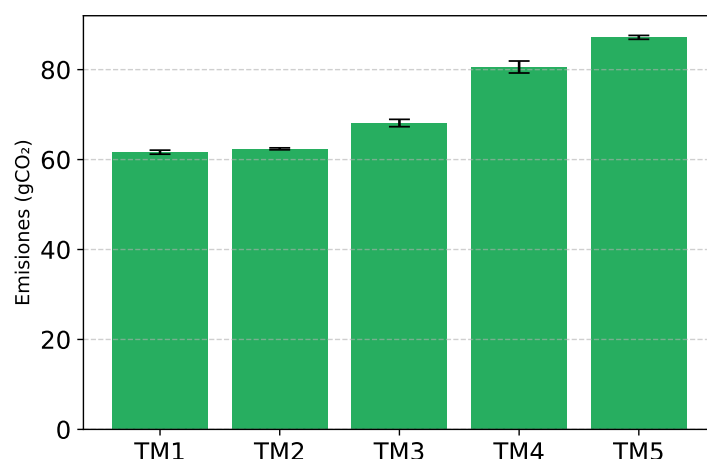


Figura 6.5: Emisiones de Abilene tras ejecutar PSO VCH 20 veces por TM con 200 partículas y 1200 iteraciones. Intervalos de confianza al 95 %

Fuente: Elaboración propia.

Estas mejoras al usar PSO VCH con 200 partículas y 1200 iteraciones, inevitablemente plantean diferentes cuestiones: si se reducen las partículas a 100 ¿cuánto mejoran los tiempos de ejecución? ¿Se mantendría la calidad de los resultados? ¿Y si además se reducen las iteraciones a 600? Todas estas preguntas se responden durante las dos siguientes pruebas: primero, se redujeron las partículas a 100 manteniendo las 1200 iteraciones, y posteriormente, las iteraciones se fijaron en 600.

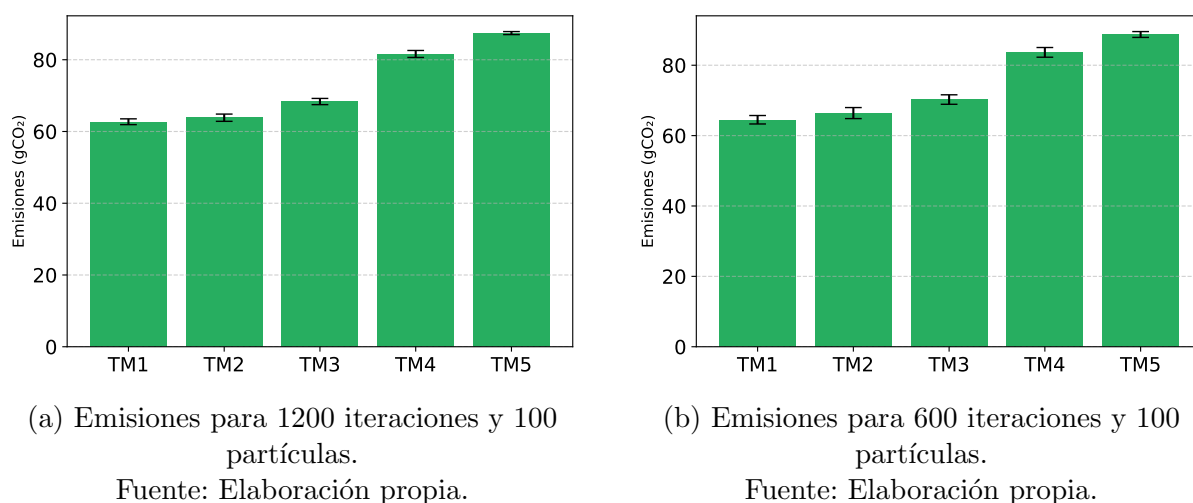


Figura 6.6: Emisiones de Abilene tras ejecutar PSO VCH 20 veces por TM. Intervalos de confianza al 95 %

Los resultados de estas dos pruebas pueden verse en la Figura 6.6. Aunque las emisiones son ligeramente más altas que con 200 partículas, siguen siendo soluciones de buena calidad. Además, los intervalos de confianza están bastante ajustados, lo cual indica que el comportamiento del algoritmo reduciendo a la mitad tanto el número de partículas como la cantidad de iteraciones, sigue presentando muy poca variabilidad y mantiene su tendencia a obtener soluciones de calidad. La mejora más significativa reside en los tiempos de ejecución, pero esto se verá en detalle más adelante en la Subsección 6.4.3.

Por último, en la Figura 6.7 se presenta una gráfica final donde se muestran los resultados obtenidos con anterioridad, así como las emisiones de Abilene con todos los enlaces encendidos. De esta manera, de un solo vistazo se pueden comparar todos los resultados y observar las mejoras que se obtienen con cada configuración.

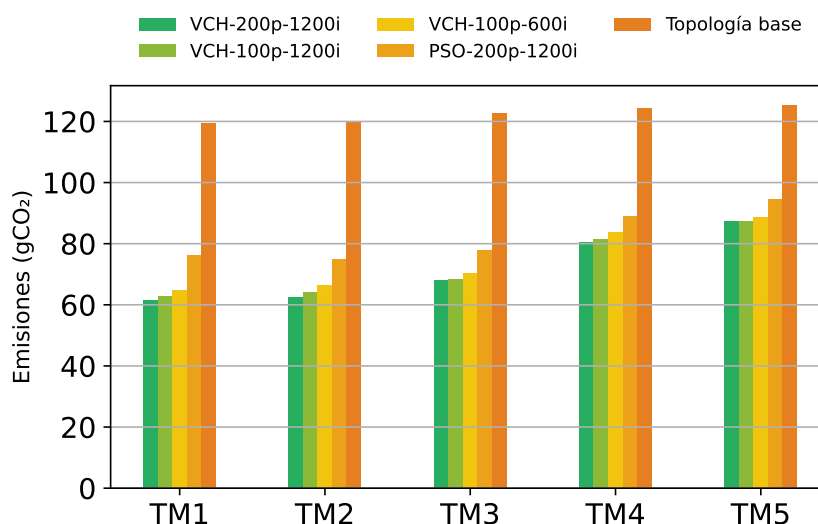


Figura 6.7: Comparación de las emisiones conseguidas en los diferentes experimentos.

Fuente: Elaboración propia.

6.4.3. Tiempos de ejecución

Un aspecto que se ha tenido muy en cuenta a la hora de realizar los experimentos es el tiempo que han tardado en ejecutarse, puesto que los escenarios con los que se tratan son muy cambiantes y requieren unos tiempos de adaptación extremadamente bajos. Este es el motivo por el cual, en la subsección anterior, se ha comentado que una de las mejoras más significativas en la reducción de partículas e iteraciones reside en el tiempo de ejecución. En la Figura 6.8 puede observarse cómo la configuración de 100 partículas y 600 iteraciones, que conseguía unos resultados de gran calidad, reducía los tiempos de ejecución en unos nueve segundos respecto a la configuración de 200 partículas y 1200 iteraciones.

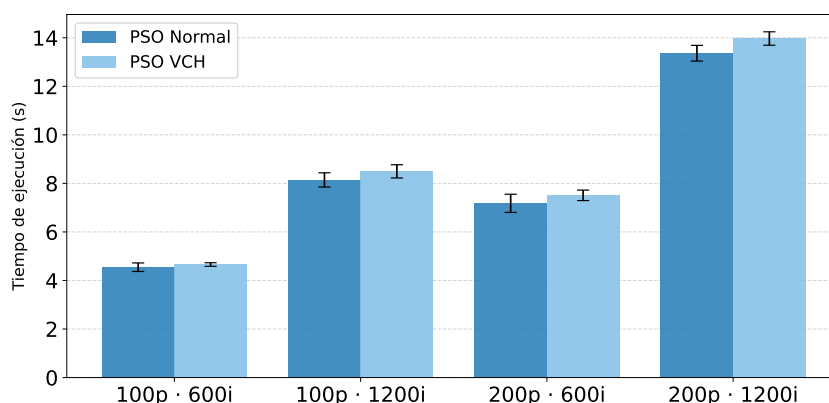


Figura 6.8: Tiempos de ejecución de Abilene. Intervalos de confianza al 95 %
Fuente: Elaboración propia.

Si se tiene en cuenta la información relativa a las emisiones que muestra la Figura 6.7 y los tiempos de ejecución de la Figura 6.8, la conclusión que se alcanza es clara: las mínimas mejoras que se obtienen al aumentar las partículas y las iteraciones no justifican duplicar o triplicar los tiempos de ejecución. Aunque, por otro lado, sí que está justificado usar PSO VCH aún tardando más que PSO normal, basta con dirigirse a la Figura 6.7 para ver que la primera estrategia presenta resultados considerablemente mejores que la segunda.

6.4.4. Porcentaje de enlaces apagados

Una métrica que resulta muy interesante estudiar es el porcentaje de enlaces que se consiguen apagar por cada matriz de tráfico. Gracias a ella, se puede saber cuánto se ha reducido la infraestructura activa de la red: cuanto mayor sea el porcentaje, mayor será la potencial reducción de las emisiones de carbono. También es un indicador de hasta qué punto el algoritmo es capaz de enrutar el tráfico usando menos enlaces sin violar las restricciones de capacidad y conectividad.

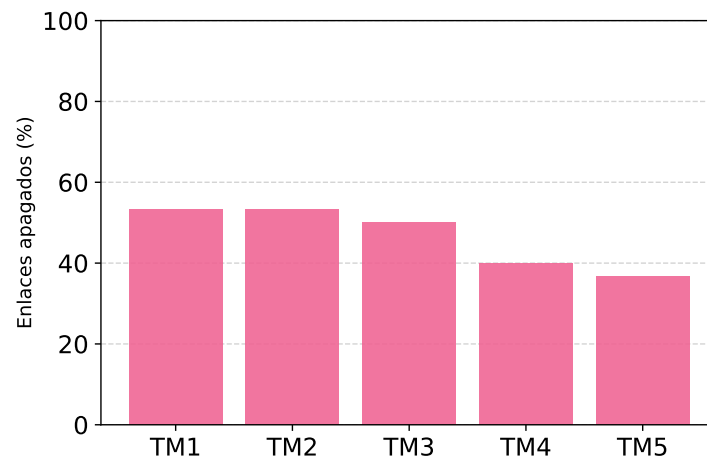


Figura 6.9: Porcentaje de enlaces de Abilene que se consiguen apagar.
Fuente: Elaboración propia.

Como se puede observar en la Figura 6.9, el algoritmo ha conseguido apagar más del 50 % de los enlaces de la red en el caso de las matrices menos cargadas como la TM1 y la TM2, mientras que al aumentar el tráfico considerablemente, como ocurre en la TM5, el algoritmo casi alcanza el 40 % enlaces apagados. En otras palabras, todas las matrices de tráfico presentan un alto porcentaje de apagado, lo que indica una gran eficiencia a la hora de redirigir el tráfico y apagar enlaces.

6.4.5. Ocupación de los enlaces tras el apagado

Medir el grado de ocupación de los enlaces que quedan encendidos tras la ejecución del algoritmo, es un gran complemento a la métrica anterior, puesto que permite conocer si los enlaces que permanecen activos están realmente aprovechados o, si por el contrario, están saturados o infrautilizados.

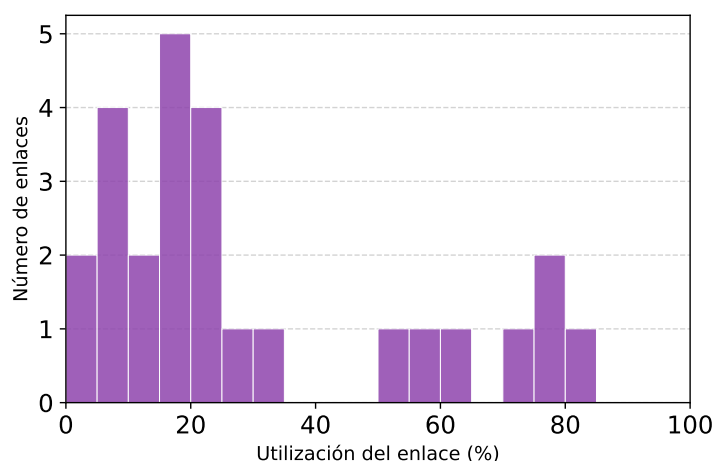


Figura 6.10: Histograma de utilización de los enlaces activos de Abilene.
Fuente: Elaboración propia.

En el histograma de la Figura 6.10 se observa que la mayoría de enlaces activos tienden a estar por debajo del 50 % de su capacidad, mientras que solo unos pocos superan el 50 %, alcanzando el 80 % de uso. Probablemente esto se deba a que el algoritmo prioriza los caminos más cortos que se encuentran con KSP, por tanto, los enlaces que pertenecen a estos caminos terminan estando mucho más utilizados que los demás.

6.5. Resultados para Nobel y Geant

Una vez se han realizado exhaustivas pruebas y experimentos con Abilene para estudiar el funcionamiento del algoritmo, llega el momento de utilizar topologías más grandes y demandantes para probar la escalabilidad del algoritmo. Para ello, se han usado las topologías Nobel-Germany y Geant, que dados sus tamaños y matrices de tráfico, son candidatos perfectos para esta tarea.

6.5.1. Emisiones de cada matriz de tráfico

De manera análoga a Abilene, para poder analizar correctamente los resultados que se obtienen de los experimentos, se muestran dos gráficas con las emisiones por cada matriz de tráfico de Geant y Nobel. De nuevo, aclarar que las gráficas no empiezan en cero, de

esta manera se aprecian perfectamente las diferencias entre las emisiones de las matrices de tráfico.

La gráfica que se muestra en la Figura 6.11 son las emisiones de carbono que genera Nobel por cada matriz de tráfico. Como ya se ha comentado con anterioridad, cuanto mayor es la matriz de tráfico, mayores serán las emisiones. Este comportamiento es justamente el que puede observarse en la gráfica, llegando a haber una diferencia de unos 10g de CO_2 entre la TM1 y la TM5.

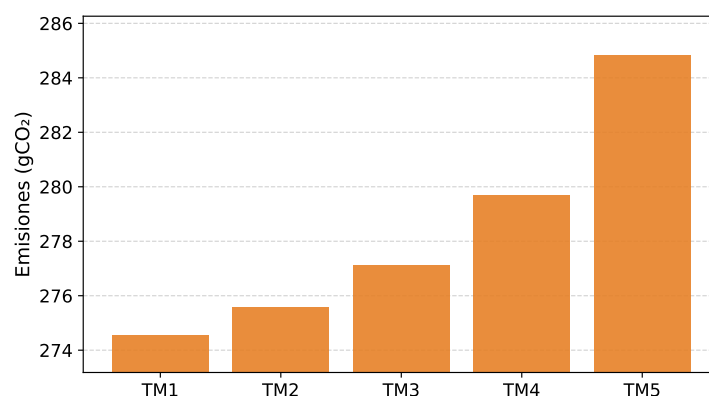


Figura 6.11: Emisiones de Nobel con todos los enlaces encendidos.

Fuente: Elaboración propia.

Geant, cuyas emisiones se encuentran en la Figura 6.12, presenta el mismo comportamiento, aunque con una diferencia de emisiones entre la TM1 y la TM5 más pequeña. De hecho, mirando las emisiones de ambas gráficas, se observa un hecho aparentemente contraintuitivo: Geant, siendo una topología más grande que Nobel, presenta menos emisiones que esta última. La razón detrás de esto es, sencillamente, que el tamaño de la red no es el único factor determinante en las emisiones, sino que también influye la intensidad de carbono asociada a la zona geográfica de los nodos. Nobel tiene todos su nodos dentro de la misma zona de emisiones, mientras que Geant los tiene repartidos por Europa y Estados Unidos, lo que le confiere una intensidad de carbono más variable.

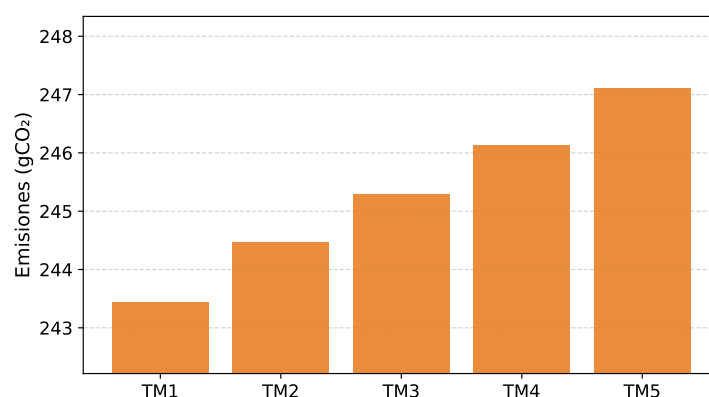


Figura 6.12: Emisiones de Geant con todos los enlaces encendidos.

Fuente: Elaboración propia.

6.5.2. Emisiones después del apagado de enlaces

Al igual que con Abilene, se ejecutan sobre Geant y Nobel las mismas configuraciones de PSO descritas en la Sección 6.4: PSO normal y PSO VCH, ambos con 200 partículas y 1200 iteraciones como configuración principal, y PSO VCH reducido a 100 partículas y 600 iteraciones. En todos los casos se realizan 20 ejecuciones por matriz de tráfico, salvo que se indique lo contrario.

Comenzando por PSO normal, los resultados de Geant se recogen en la Figura 6.13a, donde se aprecia una reducción de aproximadamente 40 gCO_2 en las tres primeras matrices de tráfico. La TM4 presenta mejoras algo menores, y la TM5 apenas experimenta variación. Nobel, en cambio, muestra mejoras más pronunciadas (Figura 6.13b), con reducciones de hasta 70 gCO_2 en las matrices 1 a 4 y de entre 30 y 40 gCO_2 en la TM5. Esta diferencia se debe al tamaño y la zona geográfica de Nobel: al tratarse de una red más pequeña con todos sus nodos en una misma zona de emisiones, la optimización se reduce a encontrar la configuración con el menor número de enlaces activos que sea capaz de redirigir todo el tráfico.

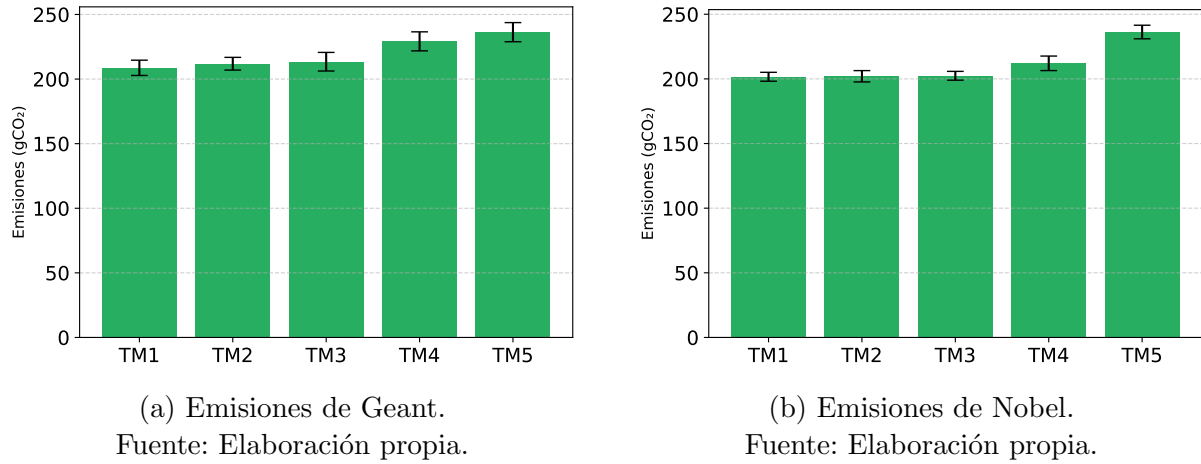


Figura 6.13: Emisiones de Geant y Nobel tras ejecutar PSO normal con 200 partículas y 1200 iteraciones 20 veces por TM. Intervalos de confianza al 95 %

Con PSO VCH y la configuración de 200 partículas y 1200 iteraciones, los resultados mejoran notablemente en ambas redes. En Geant (Figura 6.14a), las cuatro primeras matrices de tráfico consiguen reducciones de entre 70 y 80 gCO_2 respecto a las emisiones iniciales, mientras que la TM5, que apenas mejoraba con PSO normal, pasa a reducir sus emisiones en torno a 60-70 gCO_2 . En conjunto, PSO VCH logra eliminar aproximadamente un tercio de las emisiones de carbono de la red. Nobel sigue una tendencia similar (Figura 6.14b), con reducciones de entre 80 y 110 gCO_2 frente a las emisiones iniciales, lo que supone una mejora cercana al 40 %. Comparando ambas estrategias, PSO VCH supera a PSO normal en unos 25 gCO_2 de media en todas las matrices de tráfico de Nobel.

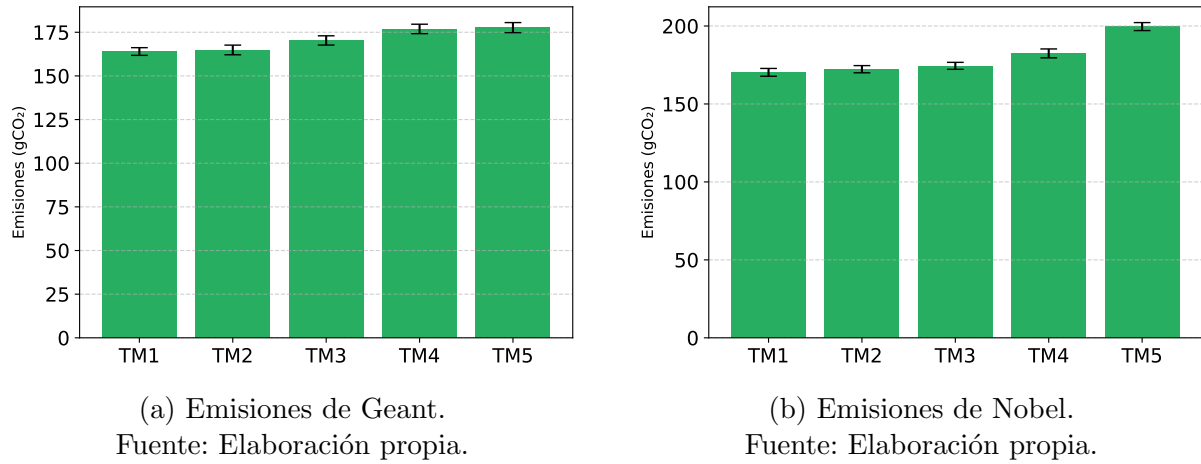


Figura 6.14: Emisiones de Geant y Nobel tras ejecutar PSO VCH con 200 partículas y 1200 iteraciones 20 veces por TM. Intervalos de confianza al 95 %

Al reducir la configuración a 100 partículas y 600 iteraciones, los resultados empeoran ligeramente, como cabía esperar dado que el espacio de búsqueda se explora con menos recursos. Aun así, las emisiones obtenidas tanto en Geant como en Nobel (Figura 6.15) siguen siendo significativamente inferiores a las de PSO normal y a las emisiones sin apagado de enlaces.

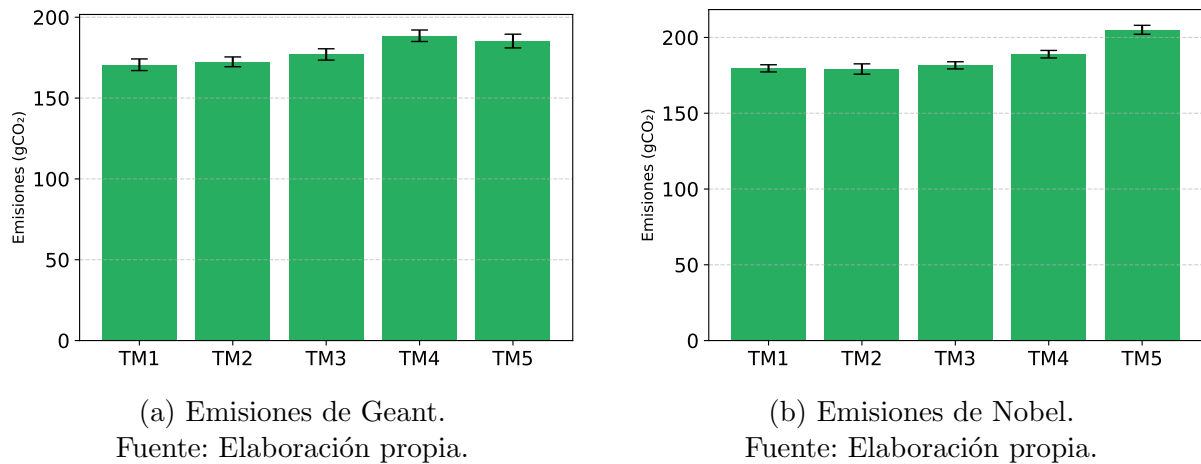


Figura 6.15: Emisiones de Geant y Nobel tras ejecutar PSO VCH con 100 partículas y 600 iteraciones. Intervalos de confianza al 95 %

Por último, la Figura 6.16 muestra una comparativa global de todas las configuraciones probadas. Ambas topologías presentan grandes mejoras en las emisiones, siendo Geant la que más se beneficia con PSO VCH, en parte por tener sus nodos en diferentes zonas de

emisiones, lo que hace que la elección de qué enlaces pagar tenga un impacto mayor en las emisiones totales.

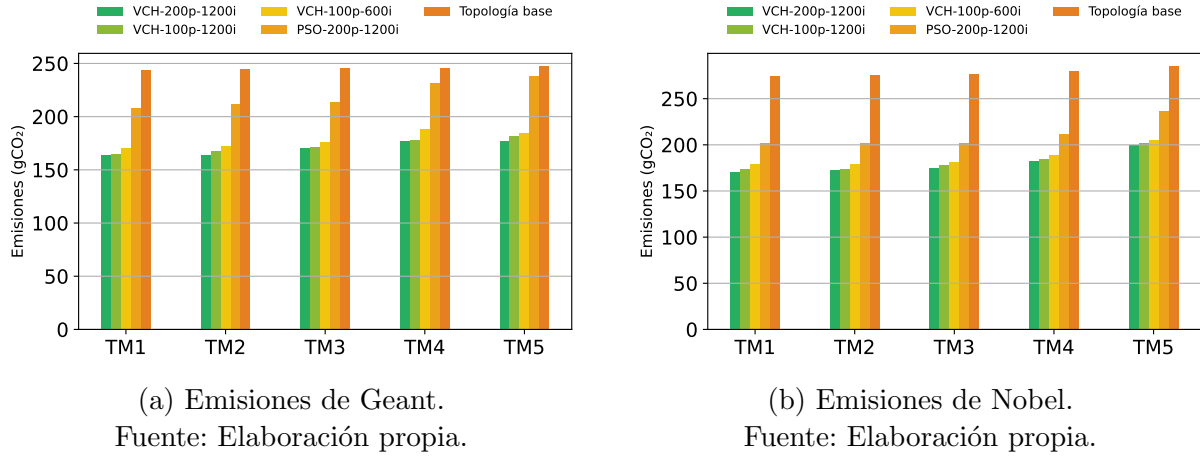


Figura 6.16: Comparación de las emisiones de Geant y Nobel conseguidas en los diferentes experimentos.

6.5.3. Tiempos de ejecución

Los tiempos de ejecución de Geant y Nobel presentan un comportamiento muy parecido al observado en Abilene. En la Figura 6.17 se presenta un gráfico con el tiempo que cada configuración del algoritmo tarda en encontrar soluciones para Geant: aumentar las partículas y las iteraciones hace que el algoritmo tarde más, pero alcanza soluciones de mayor calidad. Estos análisis de tiempo son grandes candidatos a complementarse con otros estudios, como las comparativas de la Figura 6.16a, pues permite evaluar qué configuraciones ofrecen el mejor equilibrio entre calidad y tiempo.

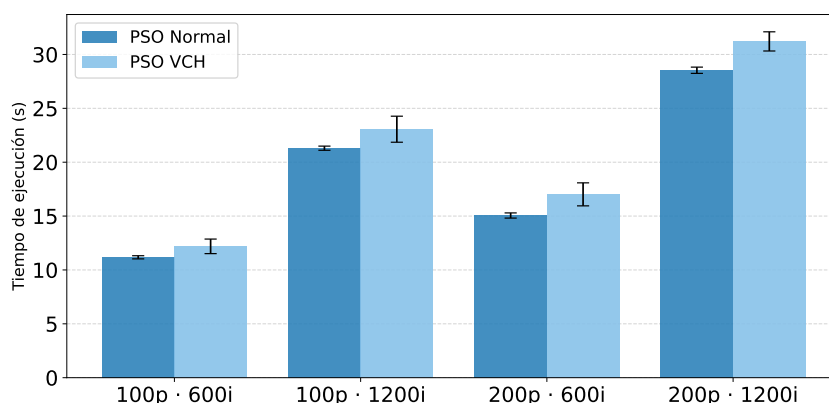


Figura 6.17: Tiempos de ejecución de Geant. Intervalos de confianza al 95 %
Fuente: Elaboración propia.

Por su parte, Nobel presenta unos tiempos ligeramente menores que Geant, tal y como puede observarse en la Figura 6.18. De nuevo, el tiempo de cómputo crece a medida que aumentan las partículas e iteraciones. Esto confirma que influye más la configuración del algoritmo que el tamaño de la red.

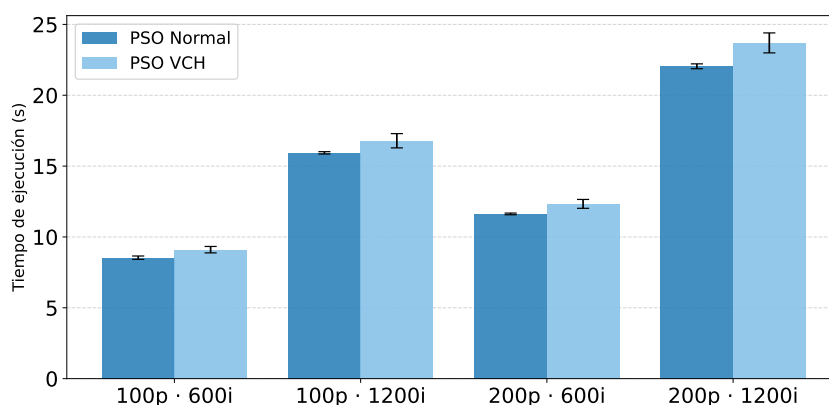


Figura 6.18: Tiempos de ejecución de Nobel. Intervalos de confianza al 95 %
Fuente: Elaboración propia.

6.5.4. Porcentaje de enlaces apagados

La Figura 6.19 muestra el porcentaje de enlaces que se han conseguido apagar en Geant por cada matriz de tráfico. En este caso, el porcentaje de enlaces apagados es menor que en Abilene, aunque sigue siendo un porcentaje bastante alto, superando el 30 % en casi todas las matrices. La TM5 es la que menos enlaces consigue apagar, lo cual es lógico,

pues al ser la más cargada, el algoritmo tiene menos margen de maniobra para redirigir el tráfico y apagar enlaces sin violar las restricciones de capacidad y conectividad. Aún así, no deja de ser un resultado muy positivo, pues conseguir apagar más del 20 % de los enlaces en una matriz tan cargada es un gran logro.

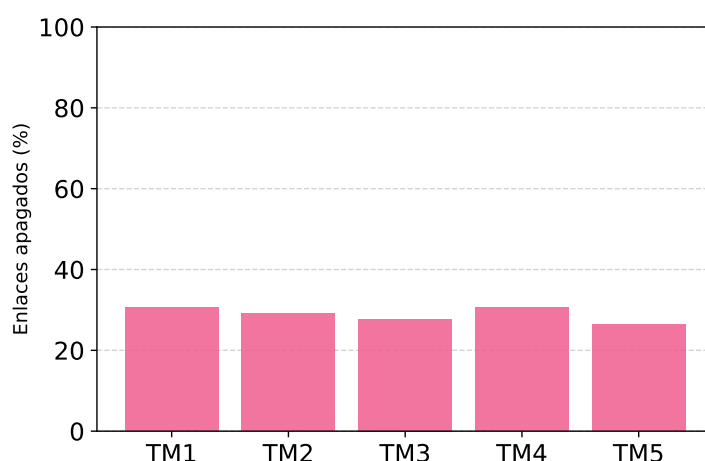


Figura 6.19: Porcentaje de enlaces de Geant que se consiguen apagar.

Fuente: Elaboración propia.

En Nobel, el porcentaje de enlaces apagados es aún mayor que en Geant, superando por poco el 40 % en las matrices de tráfico 2 y 3. La TM1 y TM4 se quedan a las puertas de superar el 40 %, mientras que la TM5, aunque es la que menos enlaces consigue apagar, supera tranquilamente el 30 %. Estos resultados son realmente buenos, pues conseguir apagar más del 30 % de los enlaces indica una gran eficiencia a la hora de redirigir el tráfico y apagar enlaces, lo que se traduce en una reducción significativa de las emisiones de carbono.

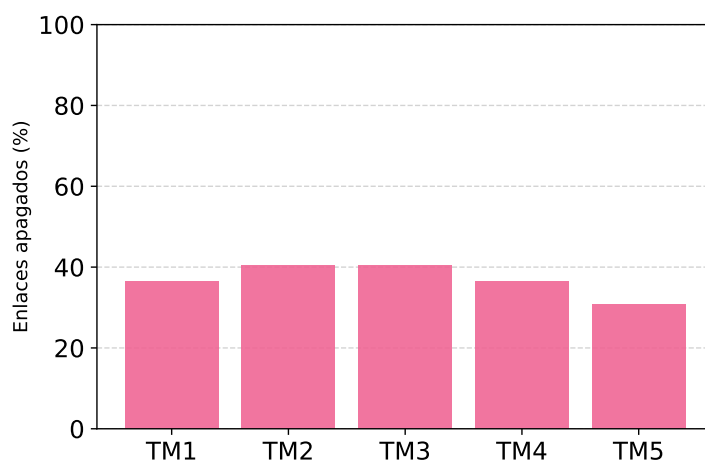


Figura 6.20: Porcentaje de enlaces de Nobel que se consiguen apagar.
Fuente: Elaboración propia.

6.5.5. Ocupación de los enlaces tras el apagado

Análogamente a Abilene, se ha estudiado el grado de ocupación de los enlaces que quedan activos tras la ejecución del algoritmo. En el caso de Geant, el histograma que se muestra en la Figura 6.21 presenta un comportamiento similar al observado en Abilene: la mayoría de enlaces activos tienden a estar por debajo del 20 % de su capacidad, mientras que solo unos pocos superan el 60 %, alcanzando el 80 % de uso. Esto indica que el algoritmo tiene margen de mejora a la hora de distribuir el tráfico entre los enlaces activos.

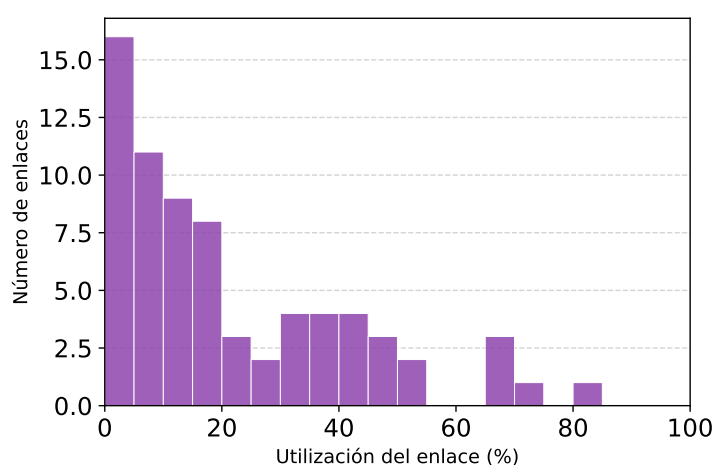


Figura 6.21: Histograma de utilización de los enlaces activos de Geant.
Fuente: Elaboración propia.

En el caso de Nobel, cuyo histograma se muestra en la Figura 6.22, el comportamiento es similar. Aunque se observa una mayor cantidad de enlaces activos con una ocupación entre el 20 % y el 40 %, la mayoría de enlaces siguen estando por debajo del 20 % de su capacidad, mientras que solo unos pocos superan el 50 %, alcanzando el 60 % de uso. De nuevo, este es un indicativo del margen de mejora que tiene el algoritmo, pues conseguir una distribución más equilibrada del tráfico entre los enlaces activos podría mejorar la calidad de las soluciones y reducir aún más las emisiones de carbono.

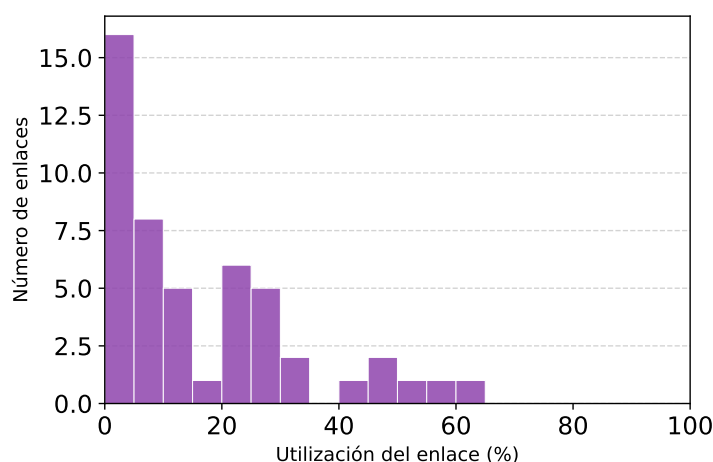


Figura 6.22: Histograma de utilización de los enlaces activos de Nobel.

Fuente: Elaboración propia.

6.6. Análisis de Germany y Limitaciones Encontradas

La topología de Germany supone un gran reto para el algoritmo, pues al tener 50 nodos y 176 enlaces unidireccionales, el espacio de búsqueda crece ampliamente. Esto le ha generado dificultades al algoritmo para encontrar soluciones de calidad, de hecho, al inicio no era capaz de encontrar ninguna solución factible. Para superar esta problemática, se fueron realizando ajustes en la configuración del algoritmo para ver cómo evolucionaban los resultados.

El primer ajuste importante fue aumentar progresivamente el número de caminos de

KSP hasta que el algoritmo arrojase soluciones factibles, lo cual ocurrió a partir de 150 caminos. Aún así, los resultados presentaban mucha variabilidad, independientemente de la cantidad de caminos que se definiesen en KSP. Finalmente, se decidió fijar el número de caminos en 500 para continuar con el estudio.

El segundo ajuste, y el más importante, fue realizar modificaciones en la inicialización de las partículas. En la Subsección 5.5.1 se comentó que, a través de diferentes pruebas, se descubrió que la aleatoriedad con la que se inicializaban las partículas afectaba a los resultados. Dichas pruebas son las que se detallan a continuación, y empezaron con leer el código de PySwarms para averiguar si se podía realizar alguna modificación u optimización que ayudase a mejorar los resultados.

Se descubrió que PySwarms permite modificar la inicialización de partículas, por lo que se realizó una primera prueba simple: una partícula con todos los enlaces encendidos y el resto de partículas con una configuración aleatoria. Esta primera prueba confirmó que guiando a PSO a través de la inicialización, se conseguían nuevos resultados, por lo que se decidió utilizar un enfoque más controlado: se mantuvo la partícula con todos los enlaces encendidos mientras que el resto de partículas se inician con entre un 50 % y un 90 % de sus enlaces encendidos.

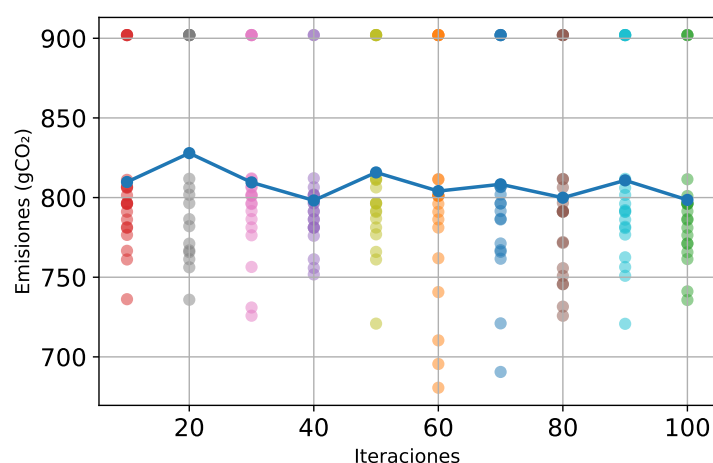


Figura 6.23: Resultados del barrido de iteraciones para la TM1 de Germany. Partículas fijas en 100.

Fuente: Elaboración propia.

Y aunque esta inicialización controlada consiguió que los resultados siempre fuesen



viables, la calidad de las soluciones seguía siendo muy variable. Para intentar corregir esta variabilidad, se decidió hacer un barrido de iteraciones fijando las partículas en 100, cuyo resultado se muestra en la Figura 6.23. Para esta gráfica se ha usado una nube de puntos con línea de tendencia, de esta manera se pueden observar las mejoras y si es capaz de converger hacia soluciones de calidad a medida que aumentan las iteraciones. Sin embargo, el comportamiento sigue siendo errático, cada nube de puntos presenta una gran dispersión que no mejora con el aumento de iteraciones. Aunque el motivo de este comportamiento no se ha podido determinar con certeza, se sospecha que el problema radica en el uso de KSP, al darle demasiada importancia a los caminos más cortos, el algoritmo termina saturando los enlaces que pertenecen a estos caminos, lo que hace que el resto de enlaces estén infrautilizados.

Capítulo 7

Conclusiones y Trabajos Futuros

A lo largo de los seis capítulos anteriores se ha detallado todo el proceso de investigación, desarrollo, implementación y evaluación del algoritmo propuesto. A partir de los conocimientos y la experiencia adquiridos en todo el proceso, en este último capítulo se recogen las conclusiones extraídas tras valorar todo el trabajo realizado, así como algunas líneas futuras de trabajo de cara a mejorar el rendimiento del algoritmo y, por ende, la calidad de sus soluciones.

7.1. Conclusiones

El objetivo principal de este trabajo ha sido proponer una solución que permita reducir las emisiones de carbono en redes cableadas. Para ello, se han investigado diferentes técnicas de optimización como ILP, DL y algoritmos metaheurísticos, de las cuales se optó por PSO, un algoritmo de optimización metaheurístico.

Una vez elegida la técnica, se diseñó e implementó un algoritmo de PSO adaptado a espacios de búsqueda discretos. La mejora de esta solución se hizo de forma iterativa a través de diferentes experimentos y evaluando los resultados obtenidos, así como investigando en profundidad el motivo de ciertos errores y funcionamientos anómalos del algoritmo.

Aunque los resultados usando topologías reales son realmente alentadores, hay que



tener en cuenta que se trata de un entorno controlado. Hay muchas variables que en este trabajo no se han tenido en cuenta, como la variación constante de la intensidad de carbono o la falta de información de los routers de las redes para modelar correctamente su consumo. También es importante conocer las características de los controladores SDN en los que se ejecutaría el algoritmo, pues afecta tanto a los tiempos de ejecución como a las emisiones finales.

A pesar de las limitaciones propias de un entorno académico, este trabajo demuestra que es posible reducir las emisiones de carbono en grandes redes combinando el enrutamiento consciente de carbono con el apagado eficiente de enlaces. A raíz de los resultados obtenidos, se podrían realizar mejoras, adaptaciones o extensiones sobre el algoritmo que se traduzcan en soluciones de mayor calidad. Es más, este trabajo podría servir como base para el rediseño de las infraestructuras de red con el objetivo de alcanzar emisiones netas cero.

7.2. Trabajos Futuros

Los análisis realizados sobre los resultados obtenidos, abren diferentes líneas de investigación y desarrollo de cara a mejorar y ampliar la solución propuesta, entre las que destacan las siguientes:

- Mejora del rendimiento computacional. Conociendo el comportamiento interno de Python, se podrían corregir los cuellos de botella del algoritmo para reducir los tiempos de ejecución sin tener que modificar la lógica del mismo.
- Ajuste dinámico de parámetros. El rendimiento de PSO depende en gran medida de sus parámetros. El uso de inteligencia artificial permitiría ajustarlos dinámicamente durante la ejecución para que el algoritmo funcione siempre con la combinación que más se ajusta a cada situación.
- Balanceo de carga en el enrutamiento. Los resultados obtenidos muestran que, tras el apagado de enlaces, la carga de tráfico se distribuye de forma desigual: mientras



que la mayoría de enlaces apenas llega al 20 % de ocupación, unos pocos alcanzan entre el 50 % y el 80 % de su capacidad. Incorporar un mecanismo de balanceo de carga en la estrategia de KSP, podría traducirse en soluciones más equilibradas y eficientes.

- Reimplementación en lenguajes de bajo nivel. Lenguajes como C, C++ o Rust ofrecen un control sobre la memoria y los recursos del sistema que Python no puede igualar, lo que se podría traducir en mejoras de rendimiento significativas y, por tanto, en la capacidad de abordar redes de mayor tamaño.

Anexos

Anexo A

Acrónimos

- IA (Inteligencia Artificial)
- SDN (*Software Defined Network*)
- PSO (*Particle Swarm Optimization*)
- QoS (*Quality of Service*)
- TFG (Trabajo de Fin de Grado)
- API (*Application Programming Interface*)
- KSP (*K-Shortest Path*)
- ILP (*Integer Linear Programming*)
- DL (*Deep Learning*)
- VCH (*Violation Constraint Handling*)
- TM (Matriz de Tráfico)
- CSV (*Comma-Separated Values*)

Bibliografía

- [1] Daniel Paska, Nina Lövehagen, Ove Persson, and Jens Malmödin. ICT energy evolution: Telecom, data centers, and AI. White paper, Ericsson, 2025. URL: <https://www.ericsson.com/en/reports-and-papers/white-papers/ict-energy-evolution-telecom-data-centers-and-ai>.
- [2] Sawsan El-Zahr, Paul Gunning, and Noa Zilberman. Exploring the benefits of carbon-aware routing. *Proceedings of the ACM on Networking*, 1(CoNEXT3):1–24, 2023. doi:10.1145/3629165.
- [3] V. Rajaraman. A concise history of the Internet. *Resonance*, 27:1841–1856, 2022. doi:10.1007/s12045-022-1483-2.
- [4] European Parliament. Resolution on the commission communication on the action plan to improve energy efficiency in the European Community. P5_TA(2001)0141, 3 2001. Document A5-0054/2001. Procedure 2000/2265(COS). URL: https://www.europarl.europa.eu/doceo/document/TA-5-2001-0141_EN.html.
- [5] Paul J. M. Havinga and Gerard J. M. Smit. Energy-efficient wireless networking for multimedia applications. *Wireless Communications and Mobile Computing*, 1(2):165–184, 2001. doi:10.1002/wcm.9.
- [6] Jose Gómez-delaHiz and Jaime Galán-Jiménez. Networking for a low-carbon future: An ILP-based carbon-aware approach for SDN. In *2025 16th International Conference on Network of the Future (NoF)*, pages 191–199, 2025. URL: <https://>

[//ieeexplore.ieee.org/abstract/document/11223308](https://ieeexplore.ieee.org/abstract/document/11223308), doi:10.1109/NoF66640.2025.11223308.

- [7] Suchismita Rout, Kshira Sagar Sahoo, Sudhansu Sekhar Patra, Bibhudatta Sahoo, and Deepak Puthal. Energy efficiency in software defined networking: A survey. *SN Computer Science*, 2(308), 2021. doi:10.1007/s42979-021-00659-9.
- [8] B. W. Boehm. A spiral model of software development and enhancement. *Computer*, 21(5):61–72, 1988. doi:10.1109/2.59.
- [9] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020. doi:10.1038/s41586-020-2649-2.
- [10] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, 2008. URL: <https://networkx.org>.
- [11] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi:10.5281/zenodo.19716234.
- [12] Lester James V. Miranda. PySwarms: a research toolkit for particle swarm optimization in Python. *Journal of Open Source Software*, 3(21):433, 2018. doi:10.21105/joss.00433.
- [13] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman,

- Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020. doi:[10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [14] Sebastian Orlowski, Roland Wessäly, Michal Pióro, and Artur Tomaszewski. SNDLIB 1.0—survivable network design library. *Networks: An International Journal*, 55(3):276–286, 2010. URL: <https://sndlib.put.poznan.pl/home.action>, doi:[10.1002/net.20371](https://doi.org/10.1002/net.20371).
- [15] ElectricityMaps. Carbon intensity data (version january 27, 2025). Online, 2025. Accessed: 2025-02-27. URL: <https://electricitymaps.com/>.